SynPEG: Synthesizing Parsing Expression Grammars via Analysis-based Pruning

Seongmin Ko and Jihyeok Park



Parsing Expression Grammars (PEGs)



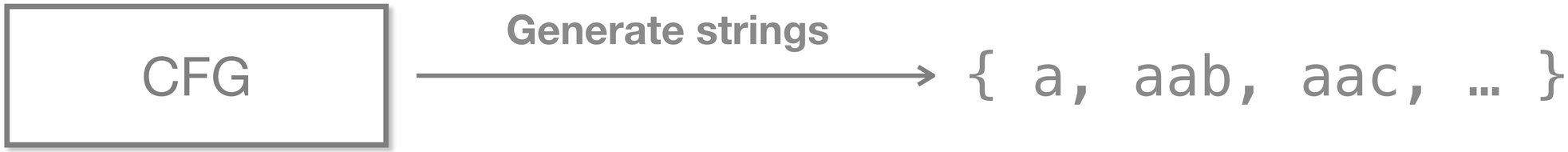
Parsing Expression Grammars:
A Recognition-Based Syntactic Foundation

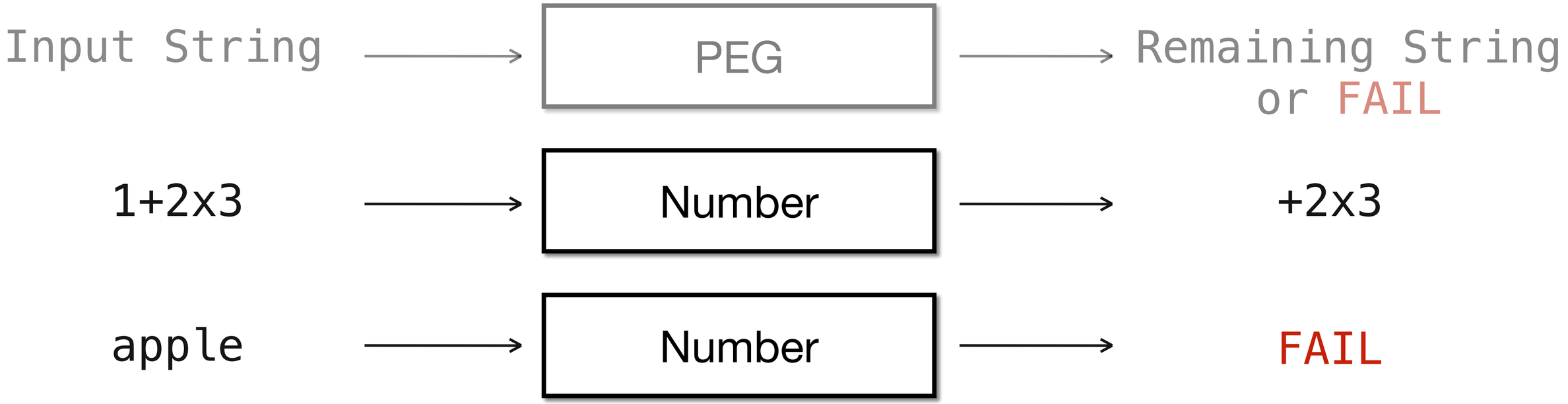
Bryan Ford
Massachusetts Institute of Technology
Cambridge, MA
baford@mit.edu

Bryan Ford, POPL 2004

A formal grammar & A parsing program

Parsing Expression Grammars (PEGs)





PEG in Practice - Python

- **Python 3.9 grammar**

```
# STARTING RULES
# ==============

file: [statements] ENDMARKER
interactive: statement_newline
eval: expressions NEWLINE* ENDMARKER
func_type: '(' [type_expressions] ')' '->' expression NEWLINE* ENDMARKER

# GENERAL STATEMENTS
# ==================

statements: statement+

statement:
    | compound_stmt
    | simple_stmts

single_compound_stmt:
    | compound_stmt

statement_newline:
    | single_compound_stmt NEWLINE
    | simple_stmts
    | NEWLINE
    | ENDMARKER
```

```
simple_stmts:
    | simple_stmt !';' NEWLINE  # Not needed, there for speedup
    | ';'.simple_stmt+ [';'] NEWLINE

# NOTE: assignment MUST precede expression, else parsing a simple assignment
# will throw a SyntaxError.
simple_stmt:
    | assignment
    | type_alias
    | star_expressions
    | return_stmt
    | import_stmt
    | raise_stmt
    | pass_stmt
    | del_stmt
    | yield_stmt
    | assert_stmt
    | break_stmt
    | continue_stmt
    | global_stmt
    | nonlocal_stmt
```

PEG in Practice - PEG Libraries

```
import scala.util.parsing.combinator.*

object ArithExpr extends RegexParsers {

  lazy val number: Parser[Number]   = "[0-9]+".r

  lazy val product: Parser[Product] = number ~ rep("x" ~> number)

  lazy val expr: Parser[Expr]       = product ~ rep("+" ~> product)
}
```



PEG in Practice - PEG Libraries

```
import scala.util.parsing.combinator.*

object ArithEx

  lazy val num

  lazy val pro                                    ~> number)

  lazy val exp                                              t)
}
```

1. Expressive and Readable Grammar
2. Deterministic, Unambiguous Parsing
3. Linear Parsing

But there are some pitfalls!



Pitfalls in Writing PEGs

In Python PEG…

```
simple_stmts:
    | simple_stmt !';' NEWLINE  # Not needed, there for speedup
    | ';'.simple_stmt+ [';'] NEWLINE

# NOTE: assignment MUST precede expression, else parsing a simple assignment
# will throw a SyntaxError.
simple_stmt:
    | assignment
    | type_alias
    | star_expressions
    | return_stmt
    | import_stmt
    | raise_stmt
    | pass_stmt
    | del_stmt
```

Order can changes parsing behavior!!

Motivation: Synthesize PEGs automatically



Programmers

write

In		Out
a	⟶	ε
ab	⟶	b
aa	⟶	FAIL
aaa	⟶	ε

Specification

SynPEG

```
A ← a(!a/aA)
```

PEG

Background: PEG Operators

Basic expressions

A ε a .

Optional and repetition

e? e* e+

Sequence

$e_1 e_2$

Predicates

&e !e

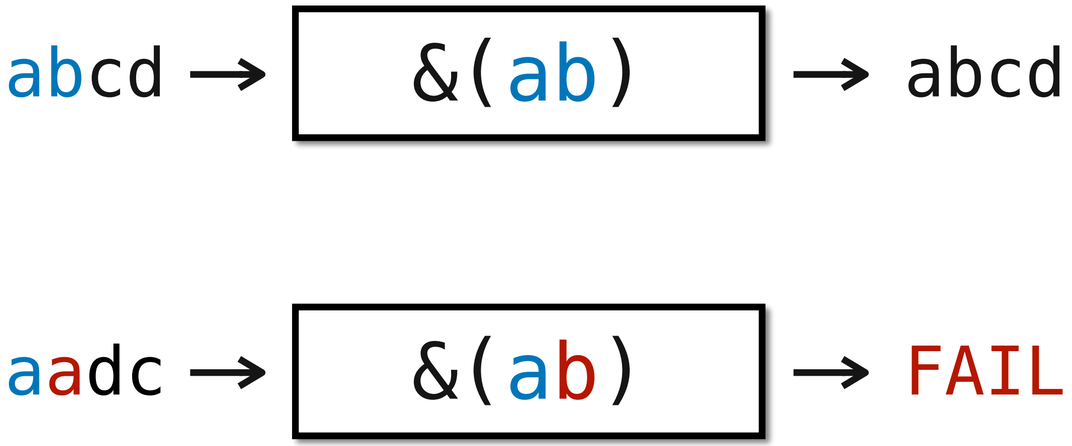
Ordered choice

e_1 / e_2

Background: Predicates

- Using PEGs as a lookahead **without consuming** input.

And-predicate

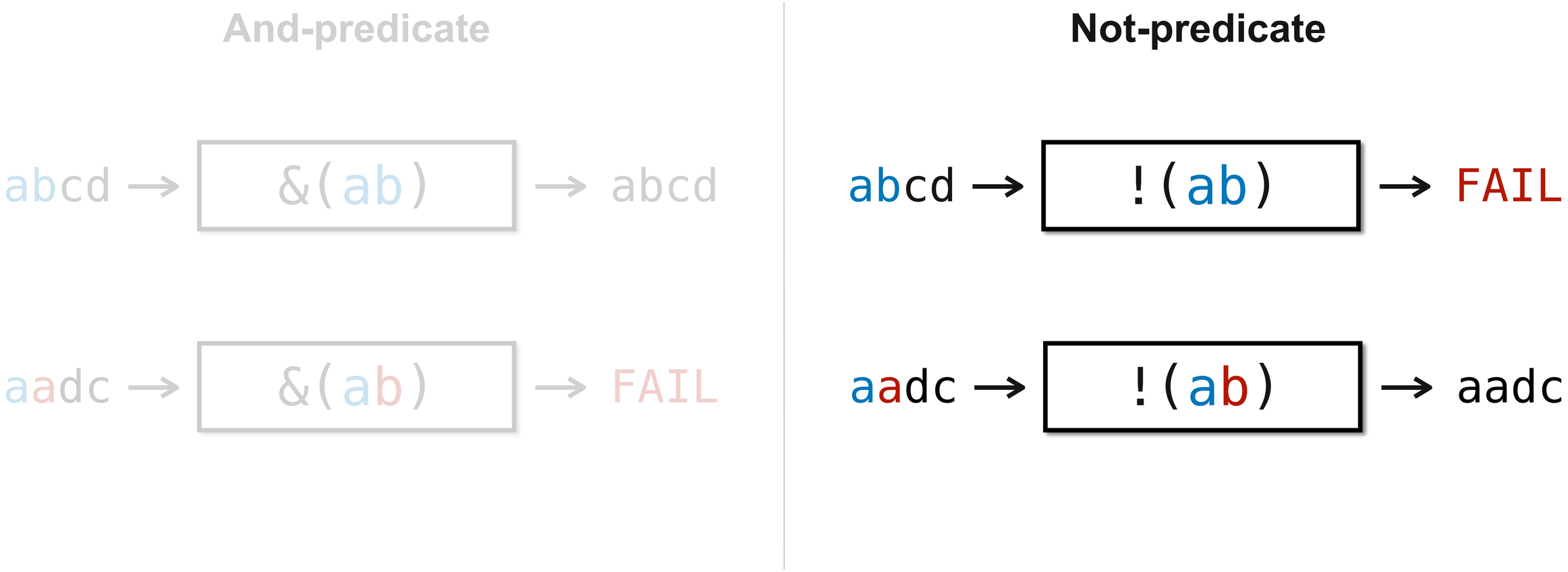


Not-predicate



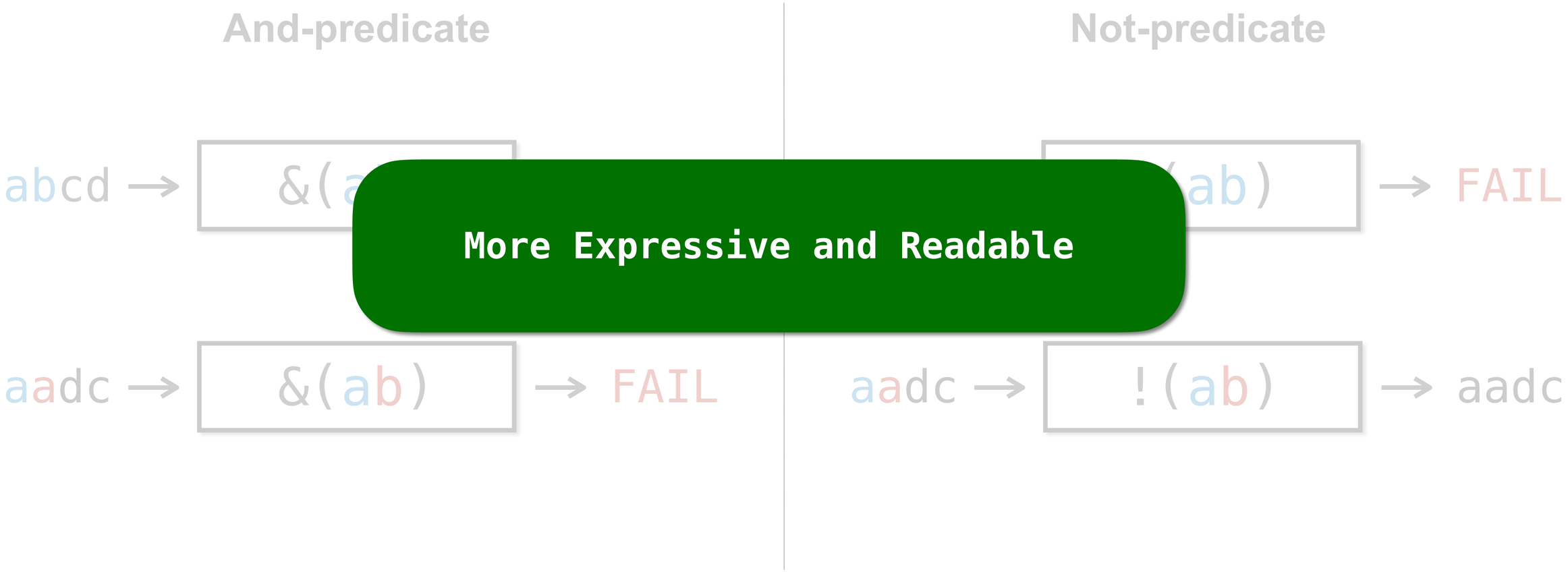
Background: Predicates

- Using PEGs as a lookahead **without consuming** input.



Background: Predicates

- Using PEGs as a lookahead **without consuming** input.



Background: Ordered Choice

- Try the left side first, and try the right side only if **left side fails**.



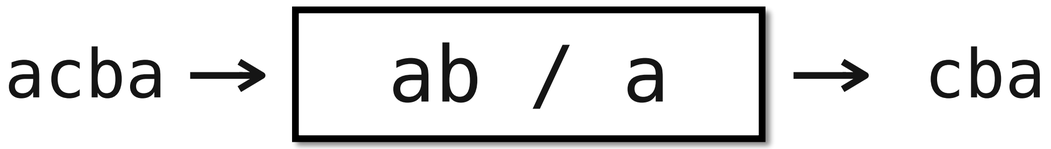
1. Try left

a	c b a

FAIL

Background: Ordered Choice

- Try the left side first, and try the right side only if **left side fails**.



1. Try left

a	c b a

FAIL

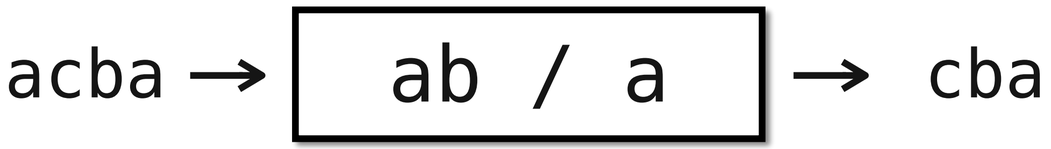
2. Left fails, then try right

a	c b a

↑

Background: Ordered Choice

- Try the left side first, and try the right side only if **left side fails**.



1. Try left

a | c b a

FAIL

2. Left fails, then try right

a | c b a

↑



1. Try left

a | b c d

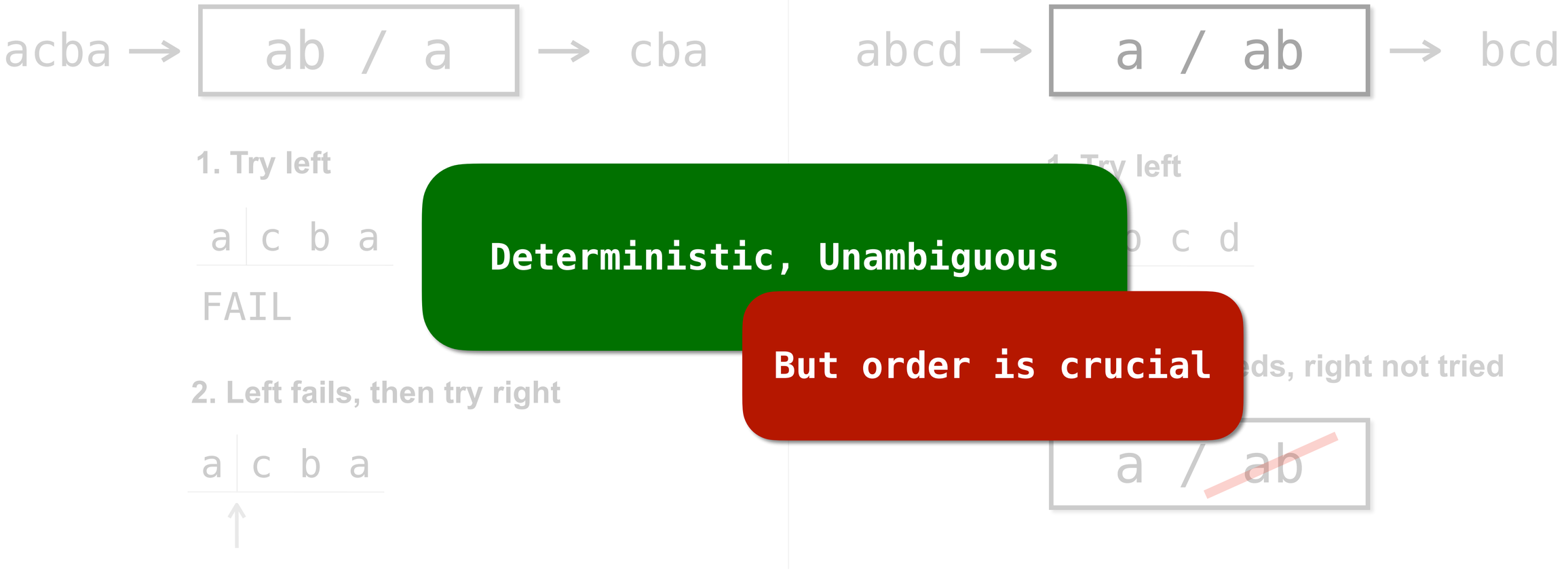
↑

2. Left succeeds, right not tried

a / ~~ab~~

Background: Ordered Choice

- Try the left side first, and try the right side only if **left side fails**.



Synthesize PEGs from Specification



Programmers

write

In		Out
a	⟶	ε
ab	⟶	b
aa	⟶	FAIL
aaa	⟶	ε

Specification

SynPEG

```
A ← a(!a/aA)
```

PEG

Running Example: Specification

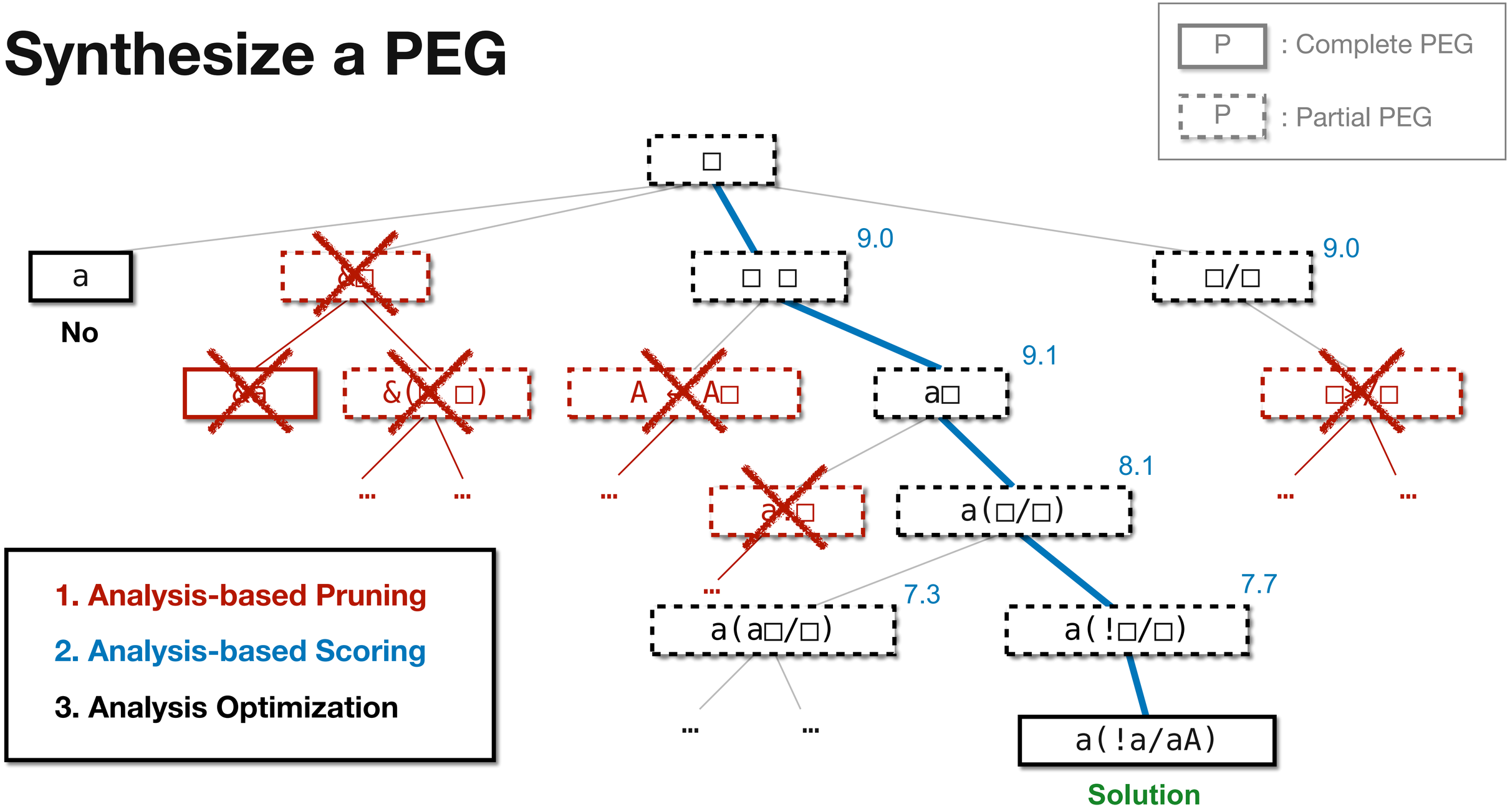
Which PEG do we want?

- Consumes as many a's as possible if the input **starts with an odd number of a**'s.
- Fails otherwise (i.e., if the input starts with an even numbers of a's).

Specification (Input-output examples)

a	⟶	ε	aa	⟶	FAIL
ab	⟶	b	bba	⟶	FAIL
aaa	⟶	ε	aaaa	⟶	FAIL
aaabbaba	⟶	bbaba	aabbab	⟶	FAIL
	...			...	

Synthesize a PEG

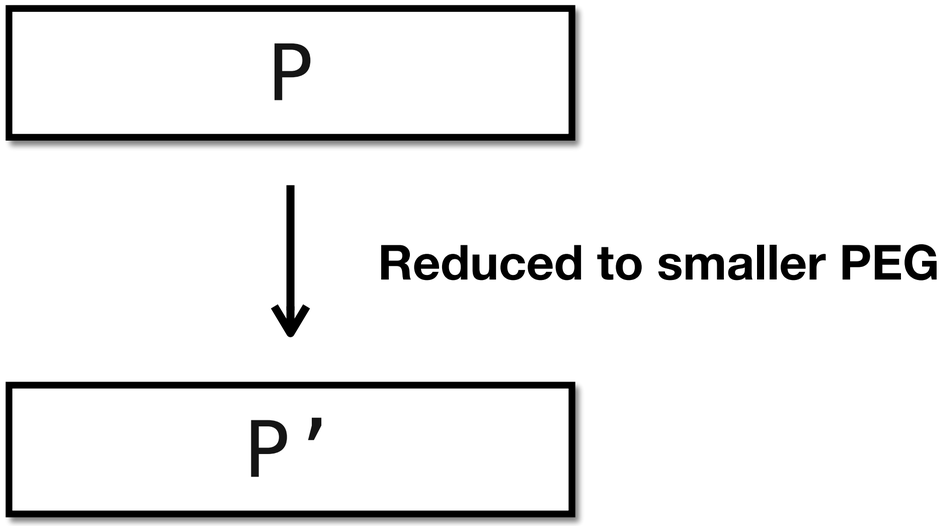


Our Approach

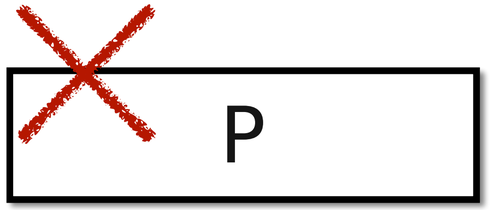
- **Analysis-based Pruning**
  1. **Grammar Analysis**
     1) Reducible PEG
     2) Left recursive PEG
  2. **Example Analysis**
     1) Invalid PEG by a single example
     2) Inconsistent PEG by multiple examples
- **Analysis-based Scoring**
- **Analysis Optimization**

Grammar Analysis-based Pruning

1. Reducible PEG



2. Left-recursive PEG

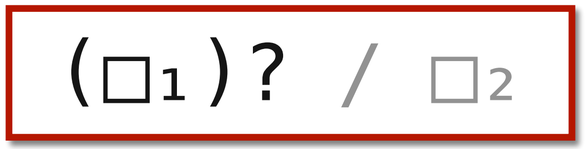


Stuck in an infinite loop

Grammar Analysis-based Pruning

1. Reducible PEG

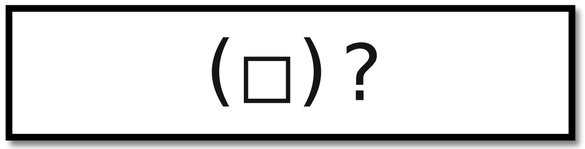
Can be pruned!



Always succeed!



Semantically Equivalent



Must-Pass/Fail Analysis

2. Left-recursive PEG

Grammar Analysis-based Pruning

1. Reducible PEG

Can be pruned!

(□1)? / □2

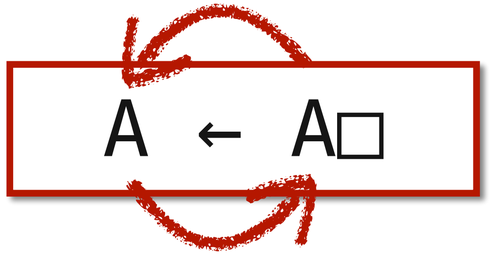
Always succeed!

Semantically Equivalent

(□)?

Must-Pass/Fail Analysis

2. Left-recursive PEG



Can be pruned!

Stuck in an infinite loop

Indirect? → Nullable Analysis

Our Approach

- **Analysis-based Pruning**
  1. Grammar Analysis
     1) Reducible PEG
     2) Left recursive PEG
  2. **Example Analysis**
     1) Invalid PEG by a single example
     2) Inconsistent PEG by multiple examples
- Analysis-based Scoring
- Analysis Optimization

Invalid PEG by a single example

Given spec

`aaabbaba → bbaba`

Partial PEG P_3

$A \leftarrow a\&(\square)$

Question

If **no expression in the hole** can satisfy the examples, can't we just **prune** it?

Invalid PEG by a single example

Given spec

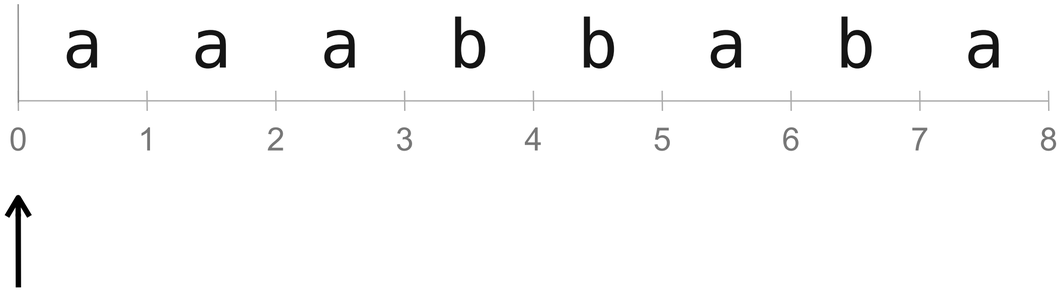
aaabbaba → bbaba

Partial PEG P_3

A ← a&(□)

↑

Parsing Step



↑

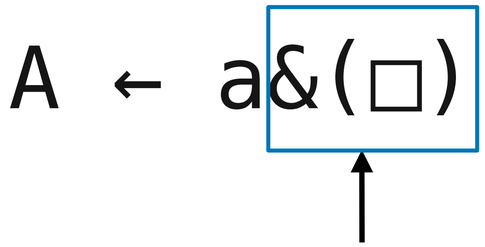
Run until the hole

Invalid PEG by a single example

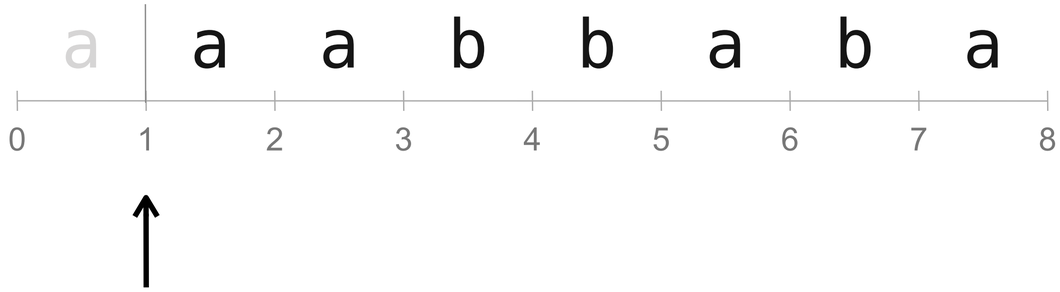
Given spec

aaabbaba → bbaba

Partial PEG P_3



Parsing Step



One a consumed. What happens next?

Invalid PEG by a single example

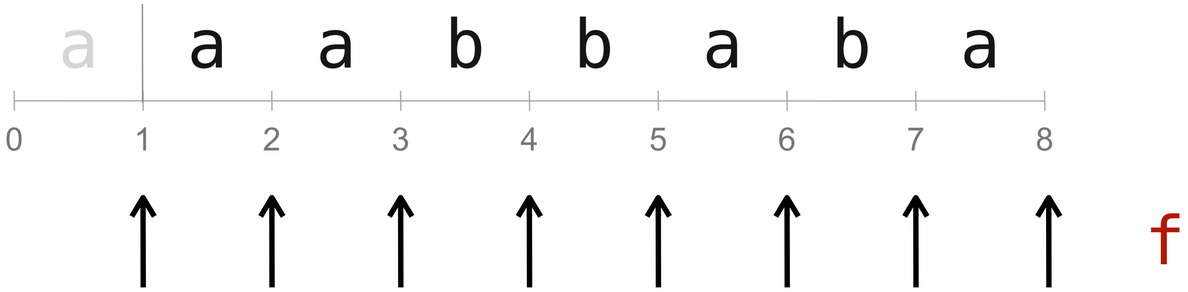
Given spec

aaabbaba → bbaba

Partial PEG P_3

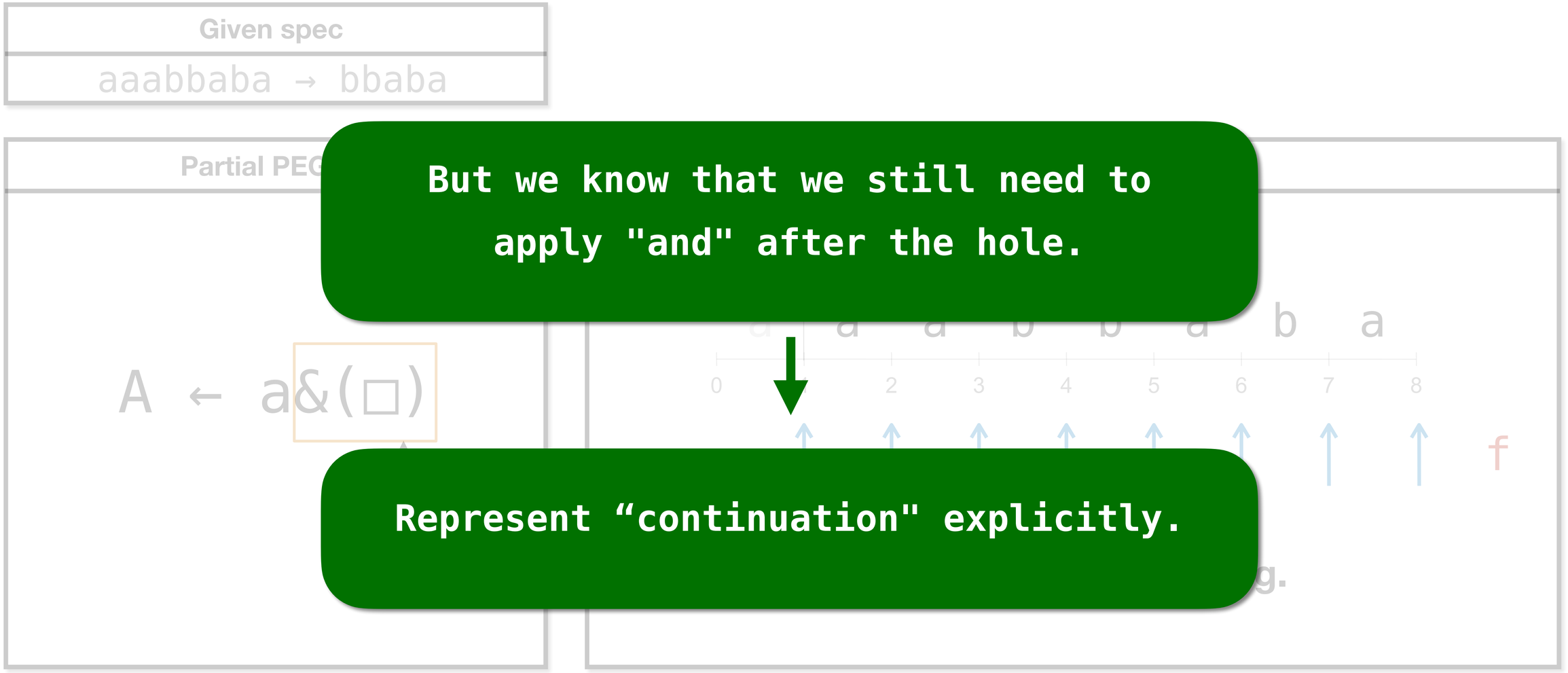
A ← a&(□)

Parsing Step



Hole can be anything.

Invalid PEG by a single example



Invalid PEG by a single example

Given spec

aaabbaba → bbaba

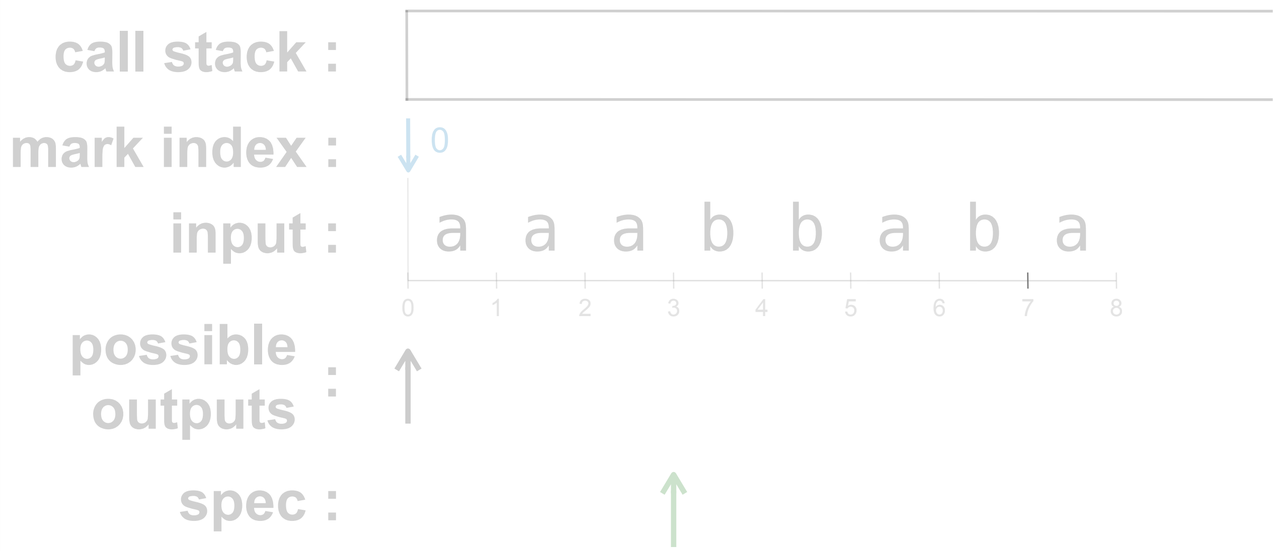
Normal Form of P_3

$A \leftarrow aB$ (↑0)

$B \leftarrow \&(\square)$

Control point ↑ active ↓ suspended

Analysis State



Invalid PEG by a single example

Given spec

aaabbaba → bbaba

Normal Form of P_3

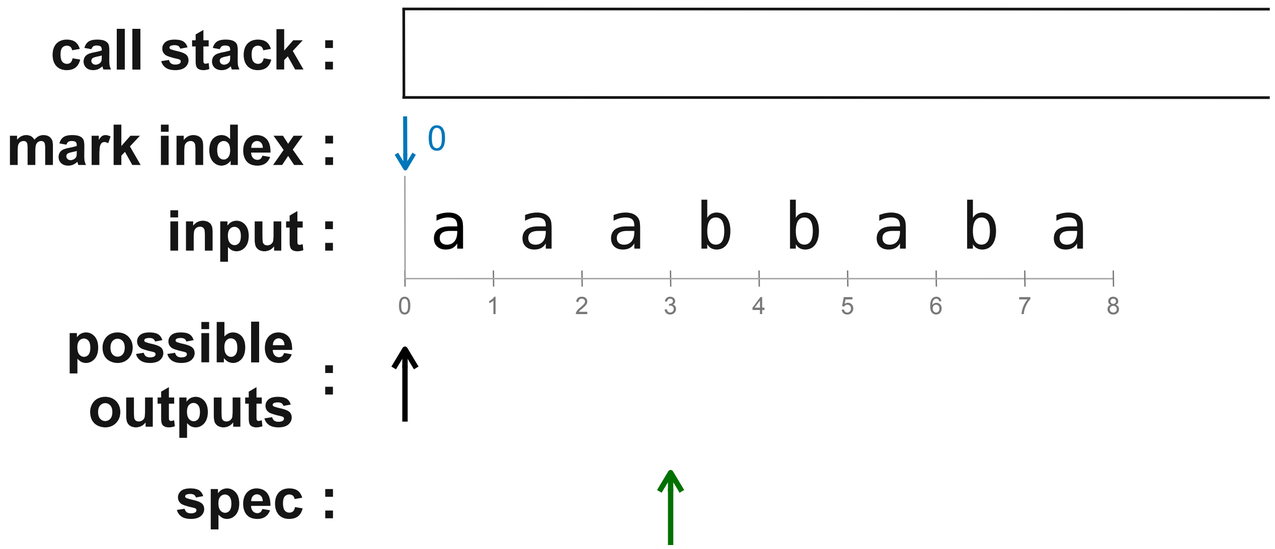
$A \leftarrow aB$

$\uparrow_0$

$B \leftarrow \&(\square)$

Control point ↑ active ↓ suspended

Analysis State



Invalid PEG by a single example

Given spec

aaabbaba → bbaba

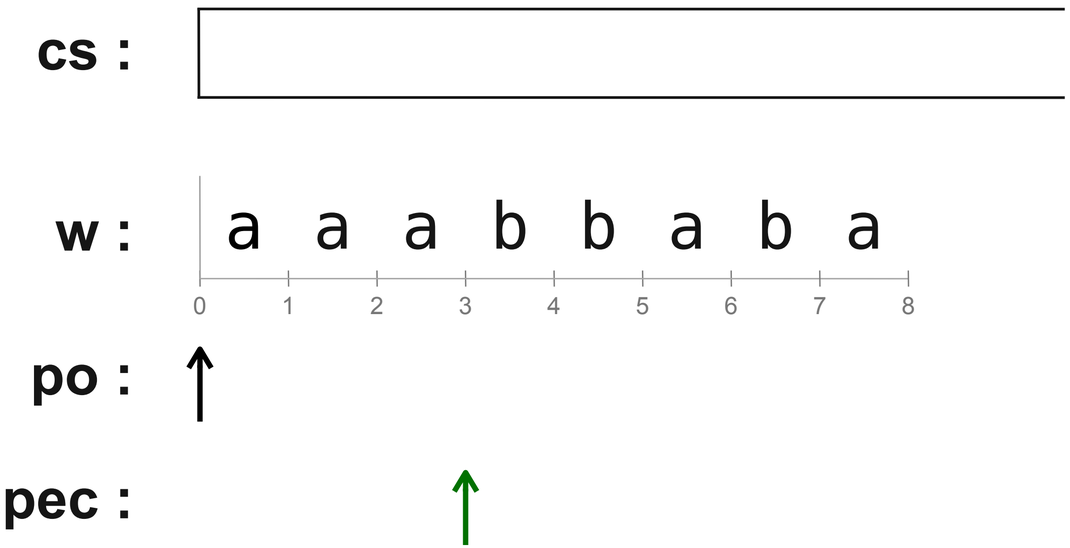
Normal Form of P_3

$A \leftarrow aB$
$\uparrow_0$

$B \leftarrow \&(\square)$

Control point ↑ active ↓ suspended

Analysis State



Invalid PEG by a single example

Given spec

aaabbaba → bbaba

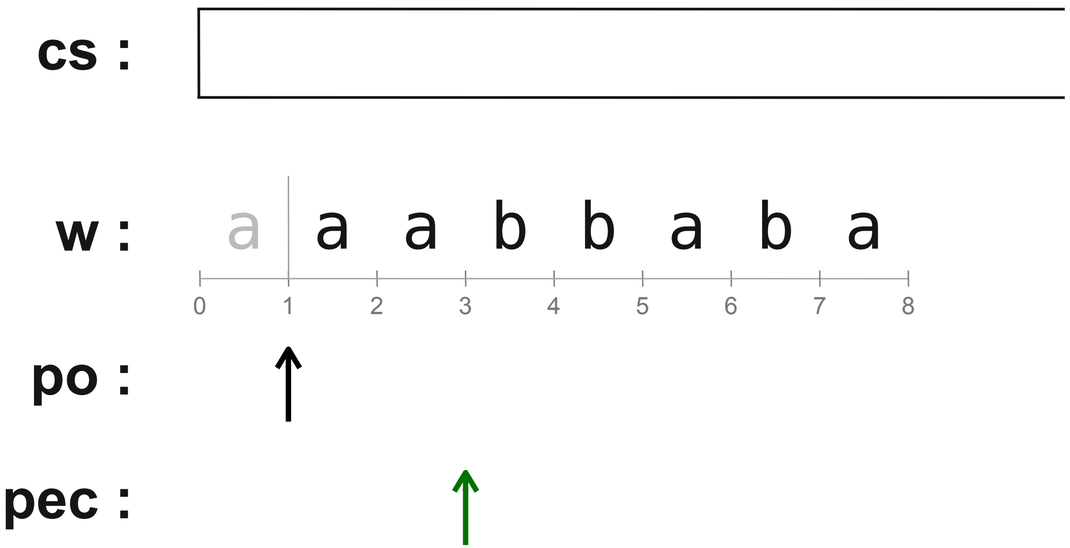
Normal Form of P_3

$A \leftarrow aB$ $\uparrow_0$

$B \leftarrow \&(\square)$

Control point ↑ active ↓ suspended

Analysis State

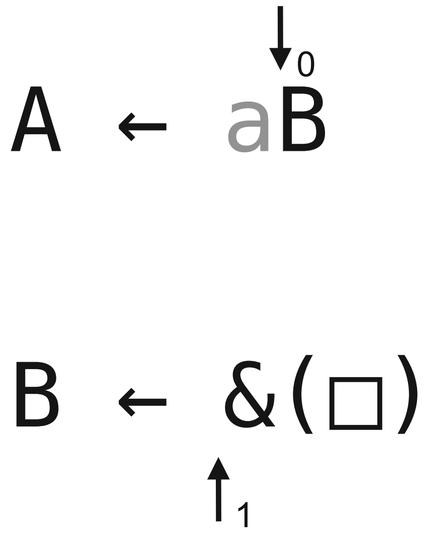


Invalid PEG by a single example

Given spec

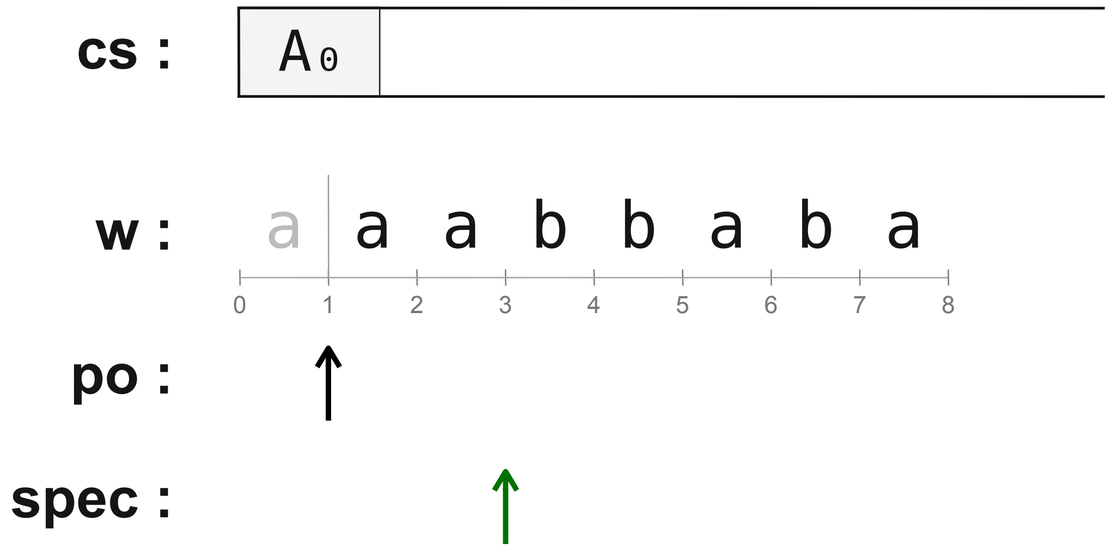
aaabbaba → bbaba

Normal Form of P_3



Control point ↑ active ↓ suspended

Analysis State

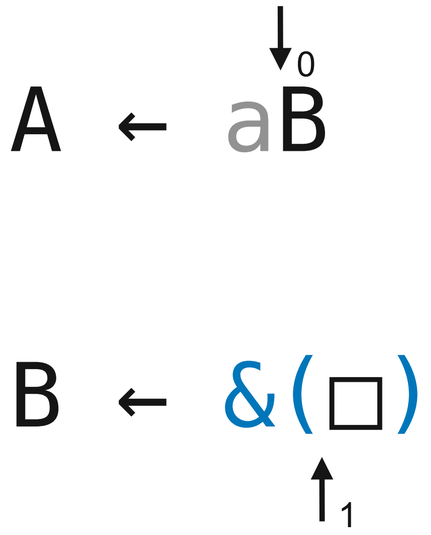


Invalid PEG by a single example

Given spec

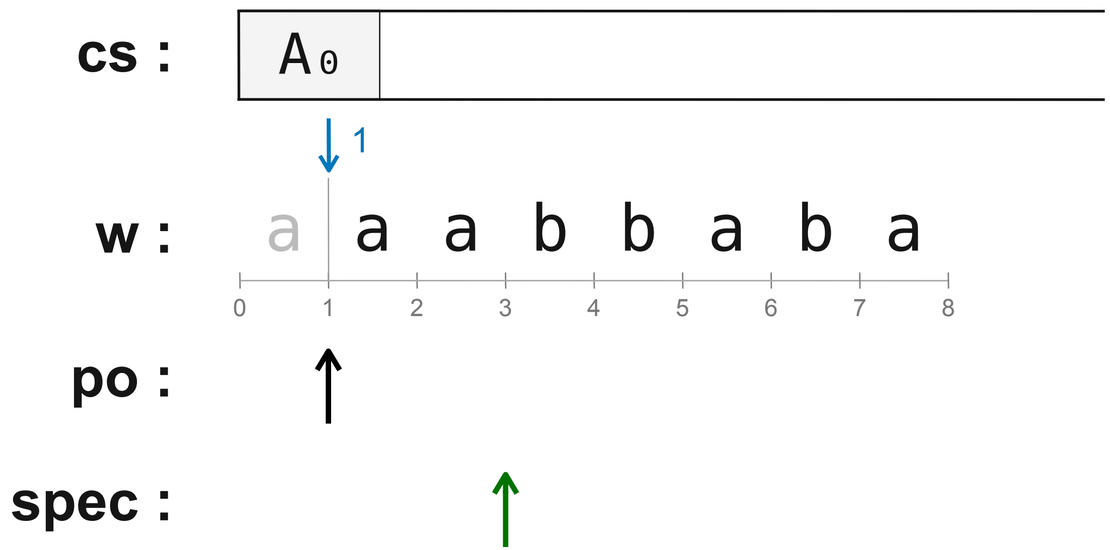
aaabbaba → bbaba

Normal Form of P_3



Control point ↑ active ↓ suspended

Analysis State

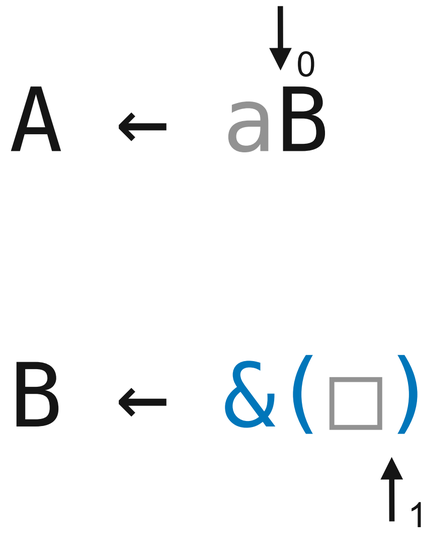


Invalid PEG by a single example

Given spec

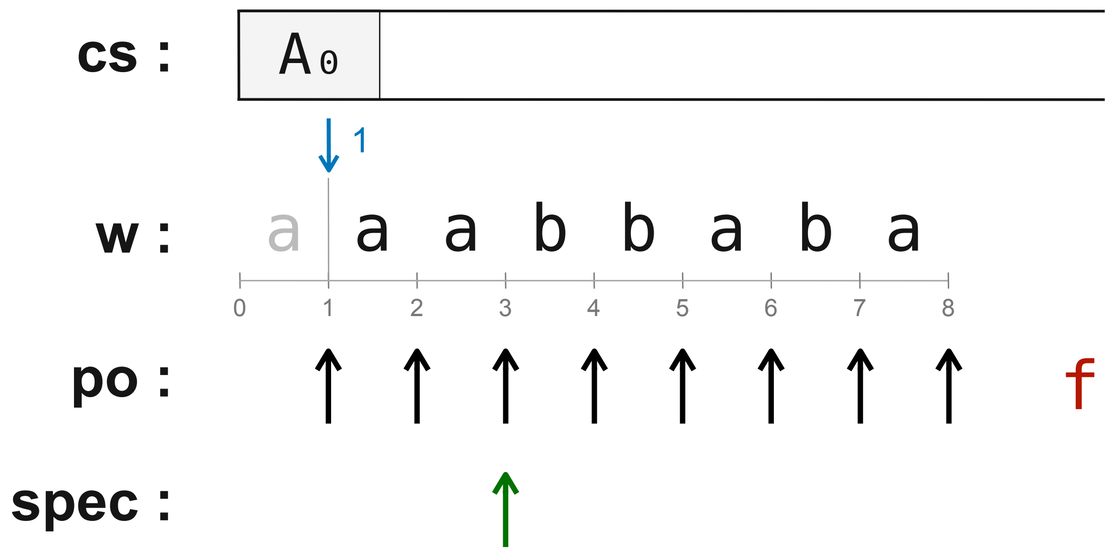
aaabbaba → bbaba

Normal Form of P_3



Control point ↑ active ↓ suspended

Analysis State

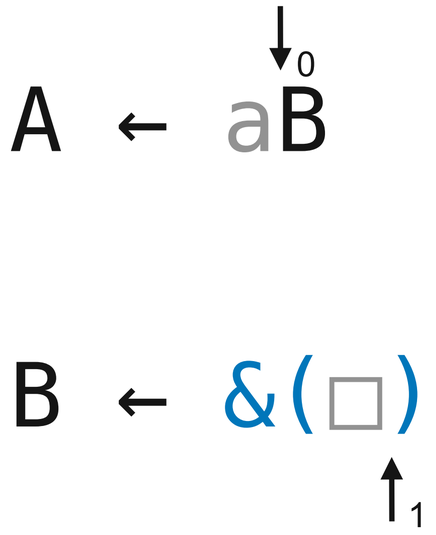


Invalid PEG by a single example

Given spec

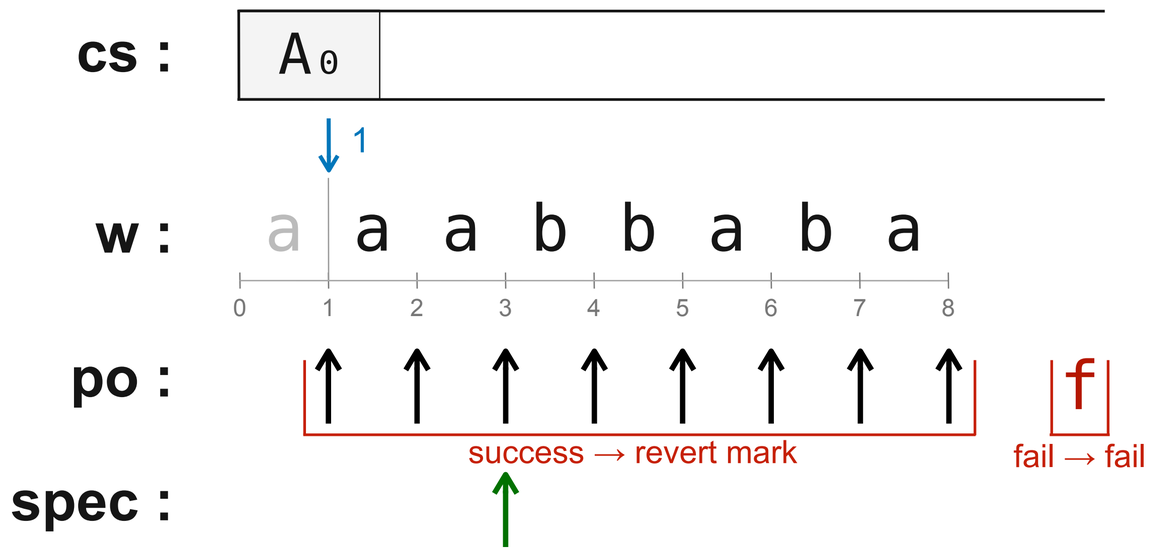
aaabbaba → bbaba

Normal Form of P_3



Control point ↑ active ↓ suspended

Analysis State

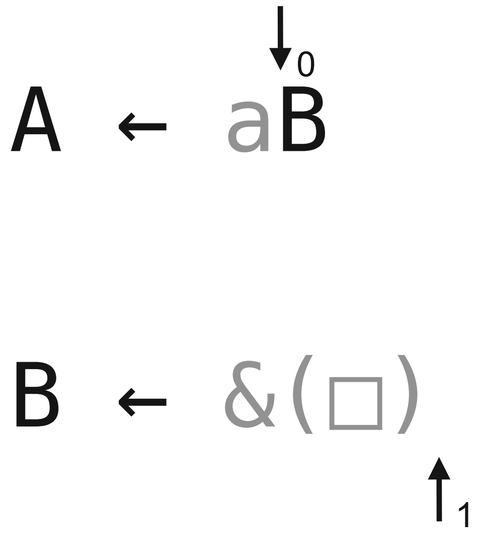


Invalid PEG by a single example

Given spec

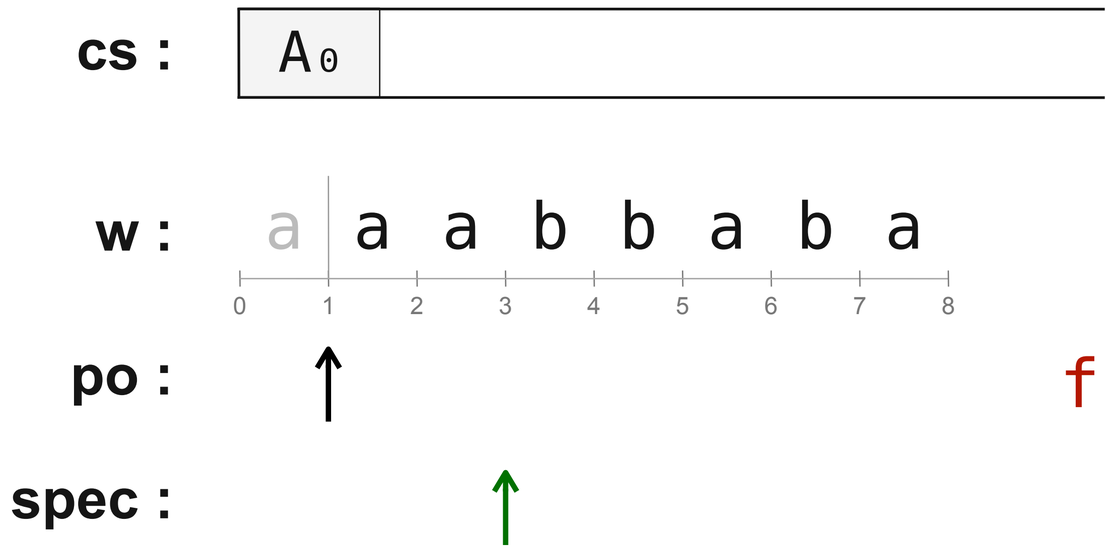
aaabbaba → bbaba

Normal Form of P_3



Control point ↑ active ↓ suspended

Analysis State



Invalid PEG by a single example

Given spec

aaabbaba → bbaba

Normal Form of P_3

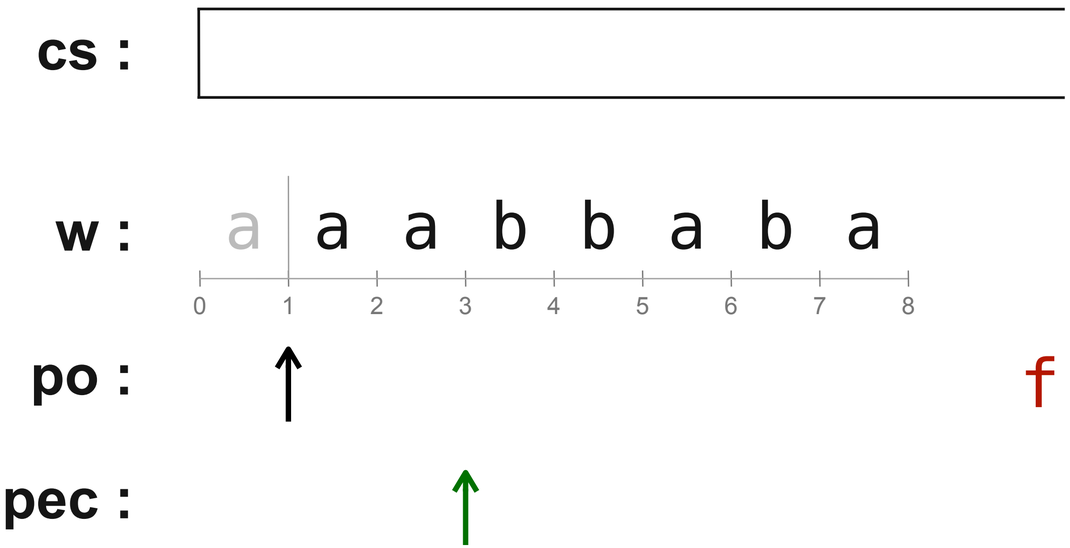
$A \leftarrow aB$

$\uparrow_0$

$B \leftarrow \&(\square)$

Control point ↑ active ↓ suspended

Analysis State



Invalid PEG by a single example

Given spec

aaabbaba → bbaba

Normal Form of P_3

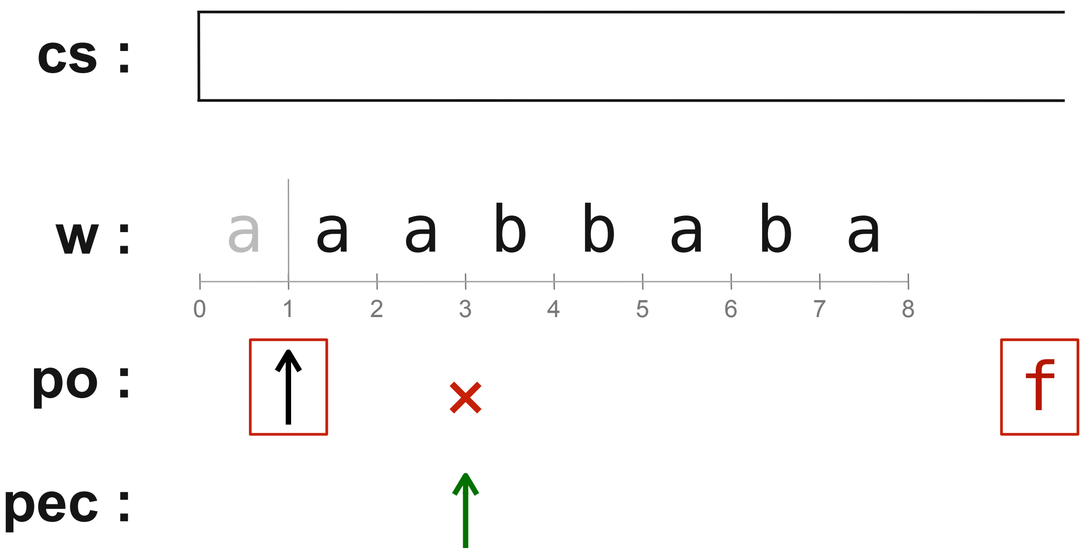
Pruning

$A \leftarrow aB$ $\uparrow_0$

$B \leftarrow \&(\square)$

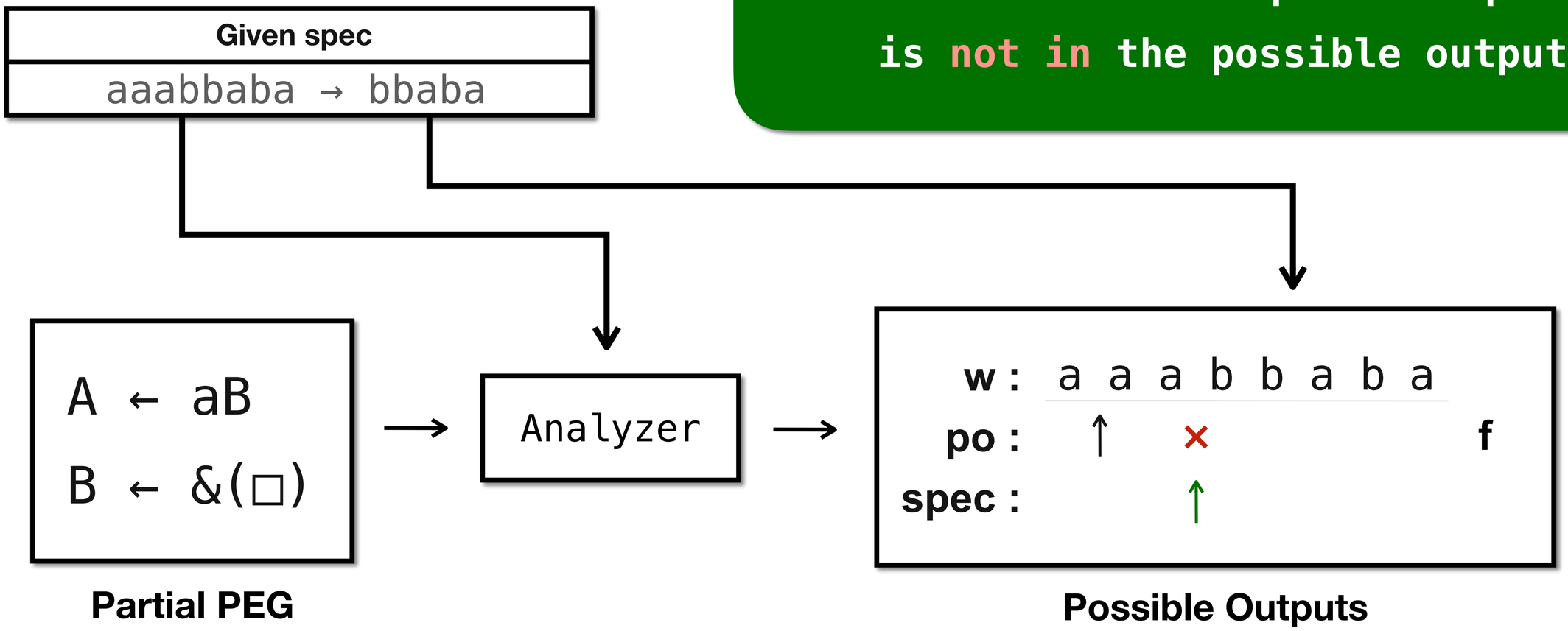
Control point ↑ active ↓ suspended

Analysis State

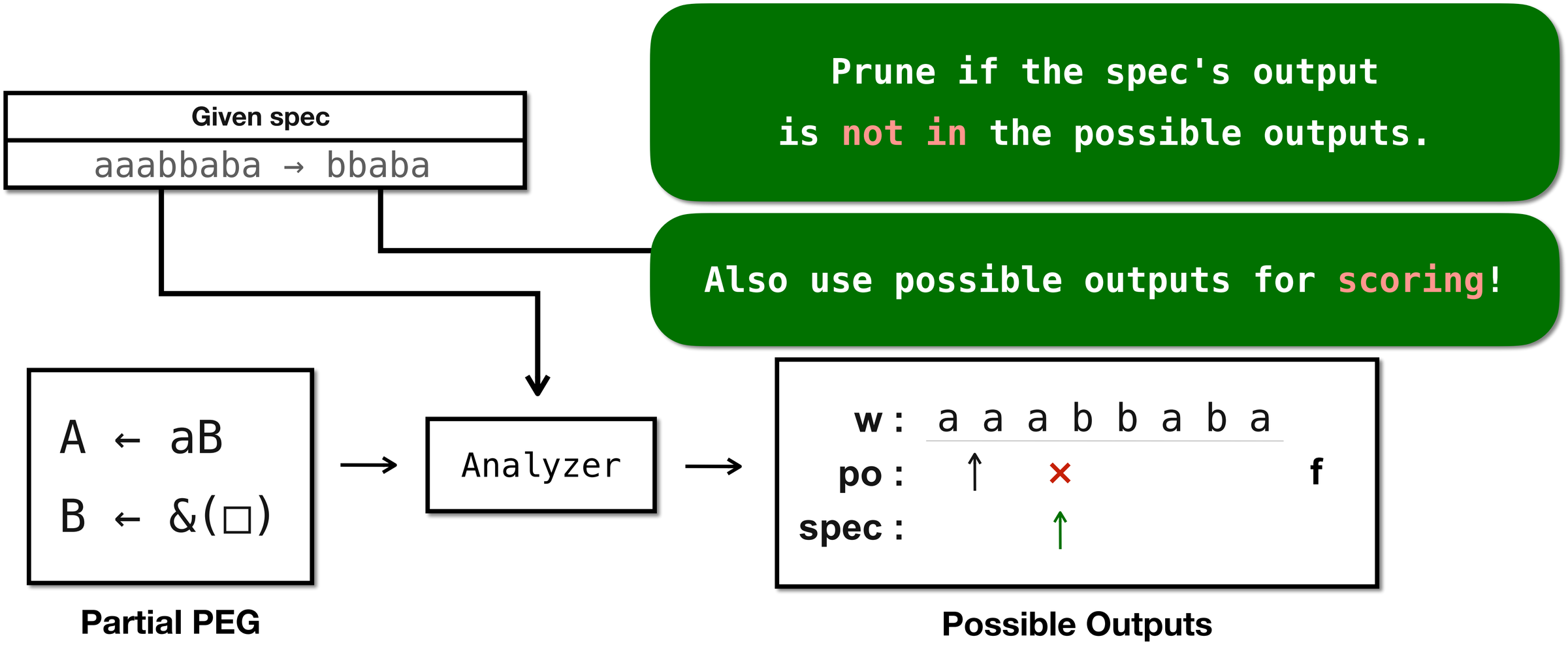


Find Possible Outputs via Analysis

Prune if the spec's output is not in the possible outputs.



Find Possible Outputs via Analysis



Inconsistent PEG by multiple examples

Given spec

```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

```
A ← BC

B ← (a)*

C ← &(D)

D ← b□
```

Control point ↑ active ↓ suspended

Question

Can any expressions satisfy both examples at the same time?

Inconsistent PEG by multiple examples

Given spec

```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

```
A ← BC

B ← (a)*

C ← &(D)

D ← b□
```

Control point ↑ active ↓ suspended

State for example 1

cs :

w : a a b b a b

po : ↑

spec : FAIL

State for example 2

cs :

w : a a a b b a b

po : ↑

spec : ↑

Inconsistent PEG by multiple examples

Given spec

```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

```
A ← BC
    ↑0
B ← (a)*

C ← &(D)

D ← b□
```

Control point ↑ active ↓ suspended

State for example 1

cs :

w : a a b b a b

po : ↑

spec : FAIL

State for example 2

cs :

w : a a a b b a b

po : ↑

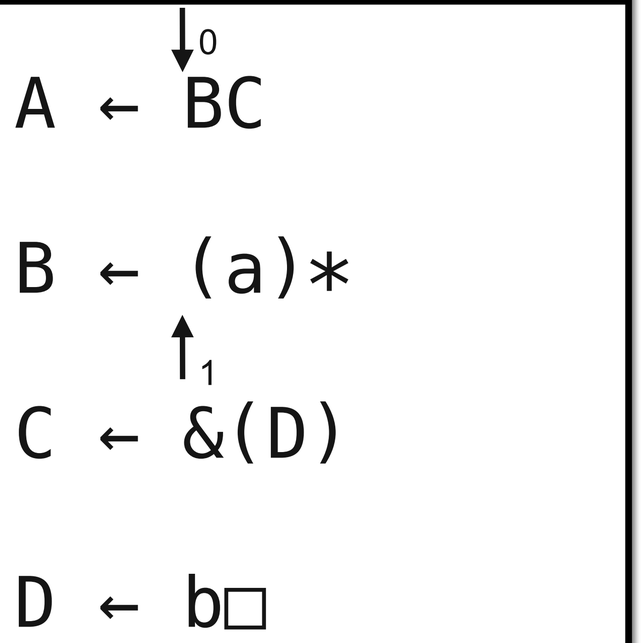
spec : ↑

Inconsistent PEG by multiple examples

Given spec

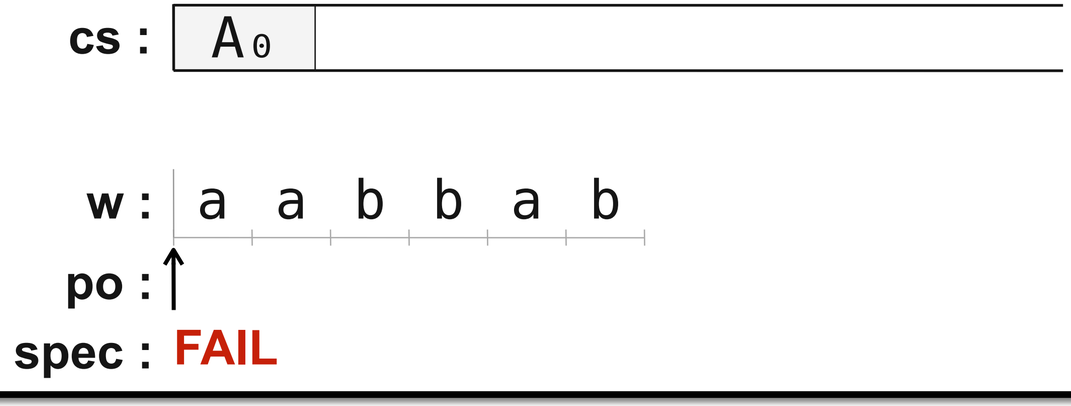
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4



Control point ↑ active ↓ suspended

State for example 1



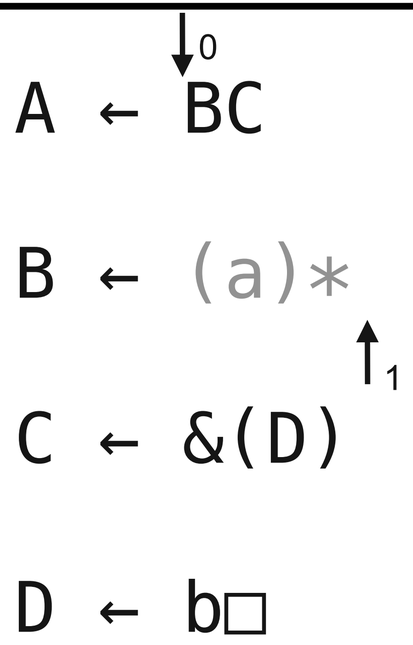
State for example 2

Inconsistent PEG by multiple examples

Given spec

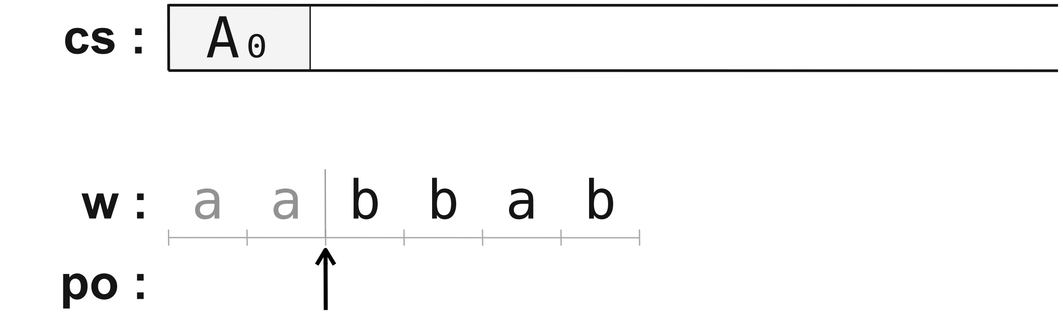
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4



Control point ↑ active ↓ suspended

State for example 1



State for example 2

Inconsistent PEG by multiple examples

Given spec

```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

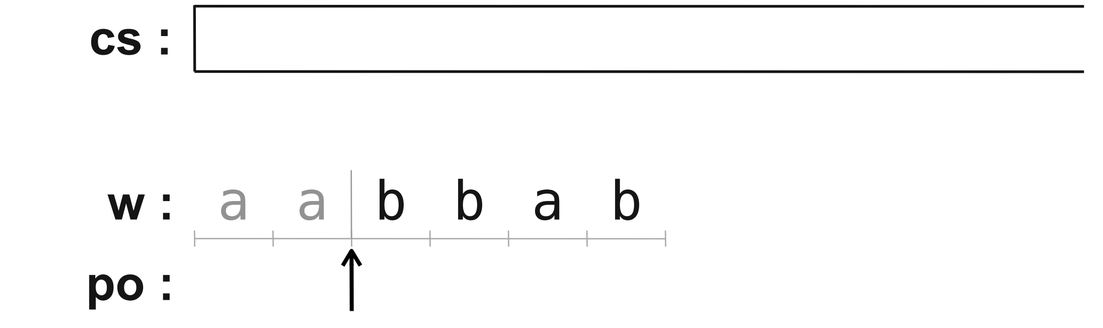
```
A ← BC
    ↑0
B ← (a)*

C ← &(D)

D ← b□
```

Control point ↑ active ↓ suspended

State for example 1



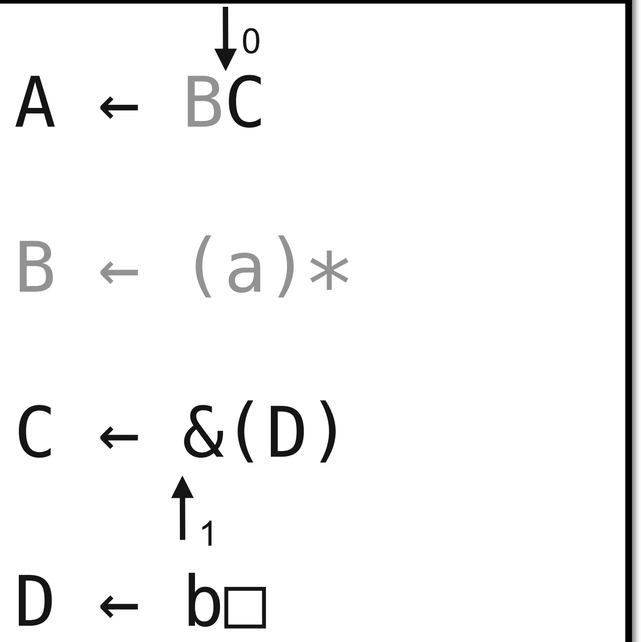
State for example 2

Inconsistent PEG by multiple examples

Given spec

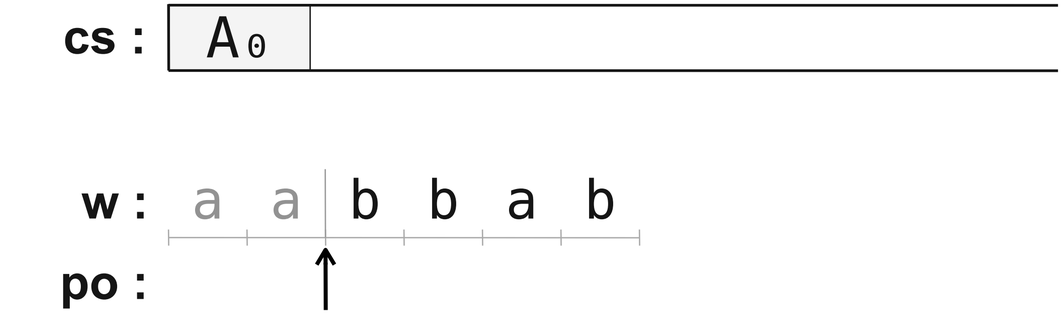
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4



Control point ↑ active ↓ suspended

State for example 1



State for example 2

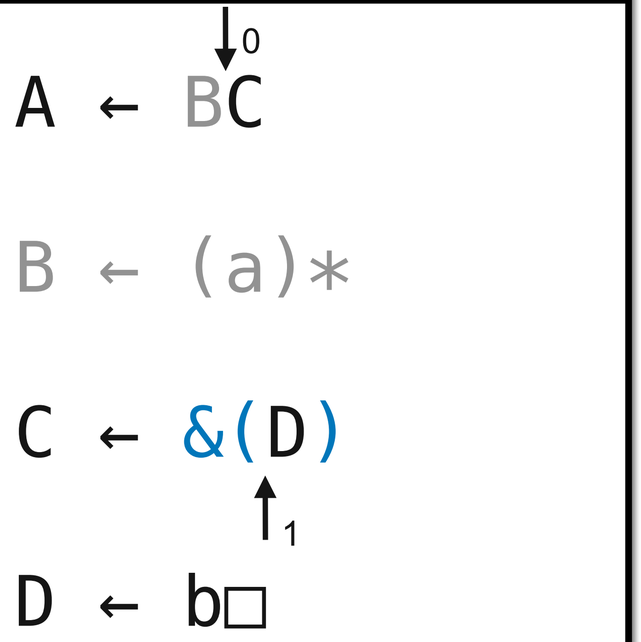
PLRG

Inconsistent PEG by multiple examples

Given spec

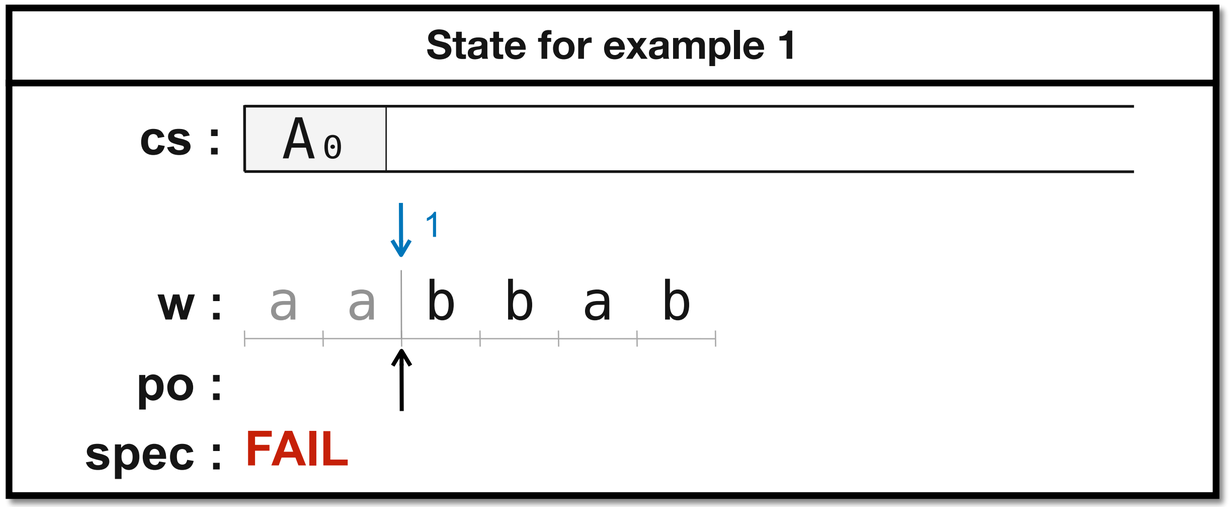
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

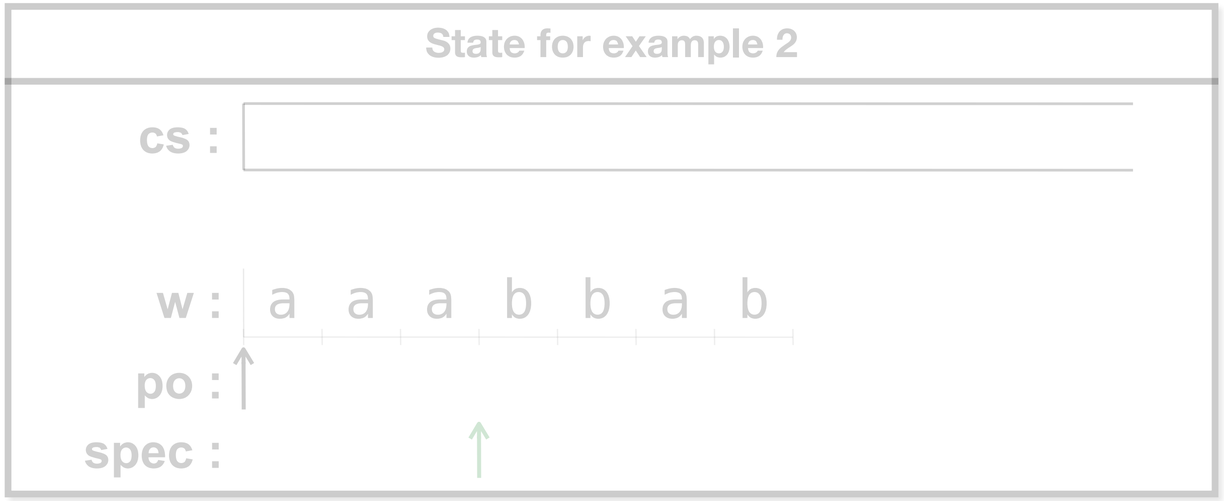


Control point ↑ active ↓ suspended

State for example 1



State for example 2

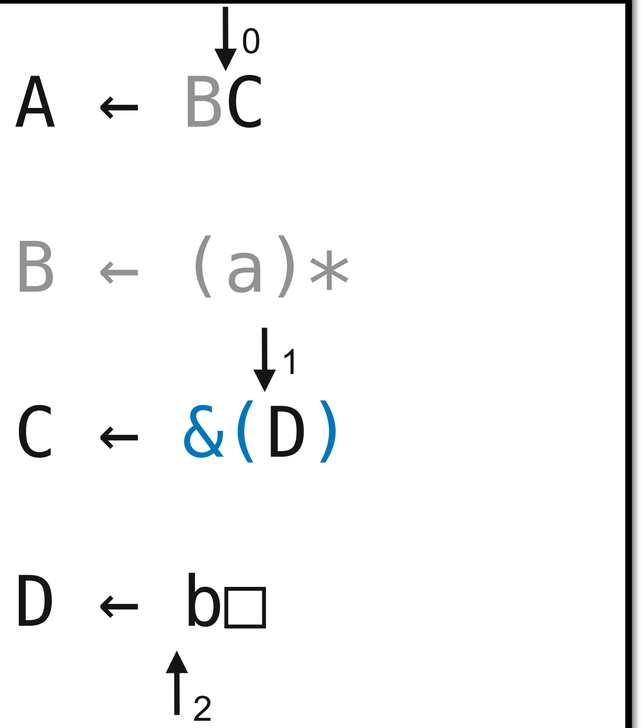


Inconsistent PEG by multiple examples

Given spec

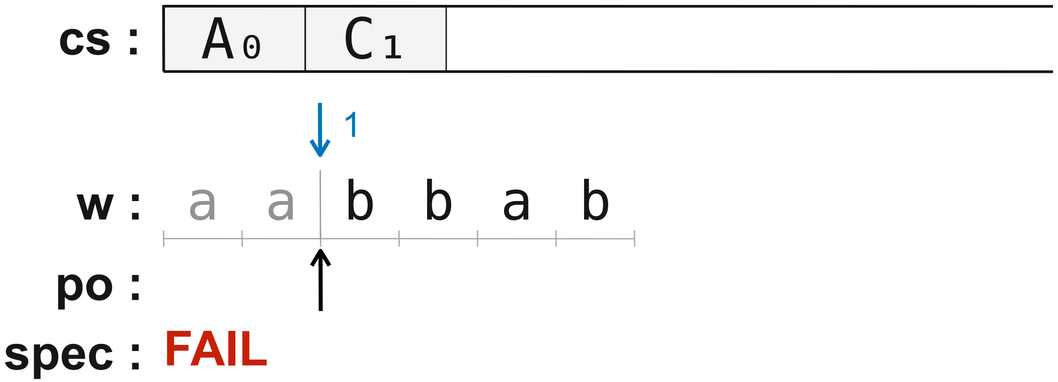
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4



Control point ↑ active ↓ suspended

State for example 1



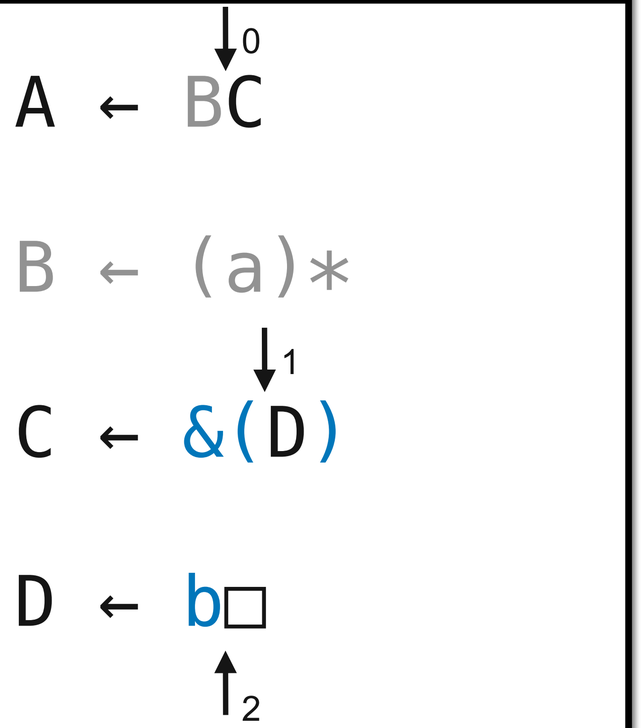
State for example 2

Inconsistent PEG by multiple examples

Given spec

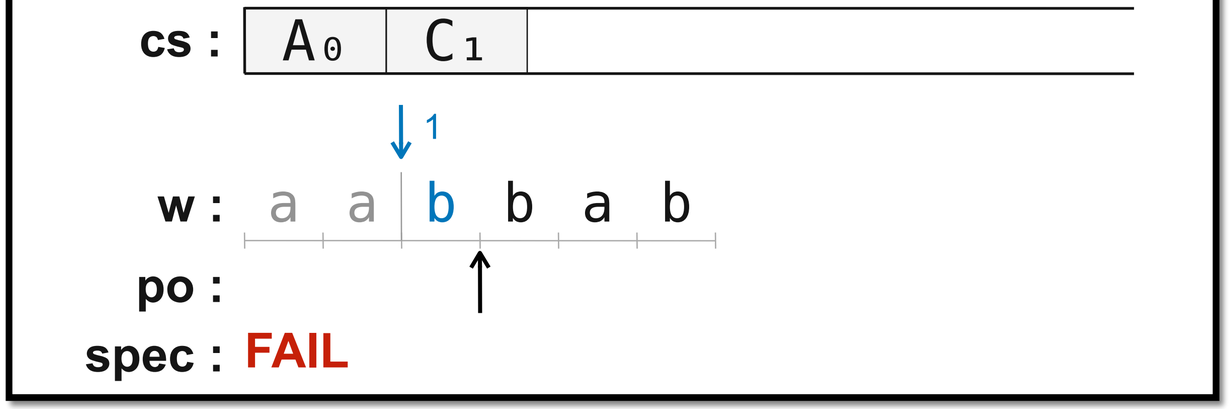
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

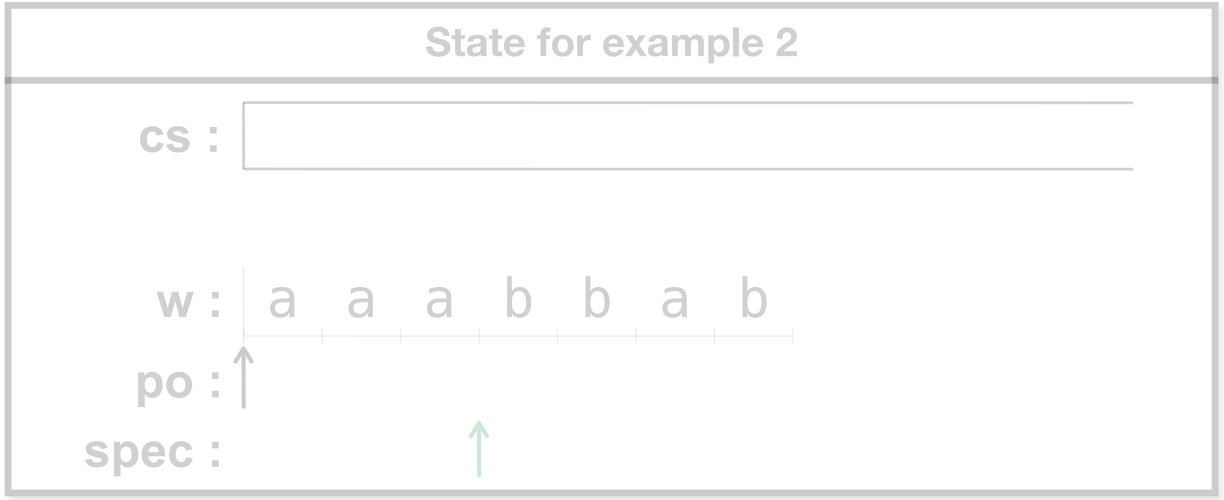


Control point ↑ active ↓ suspended

State for example 1



State for example 2



PLRG

Inconsistent PEG by multiple examples

Given spec

```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

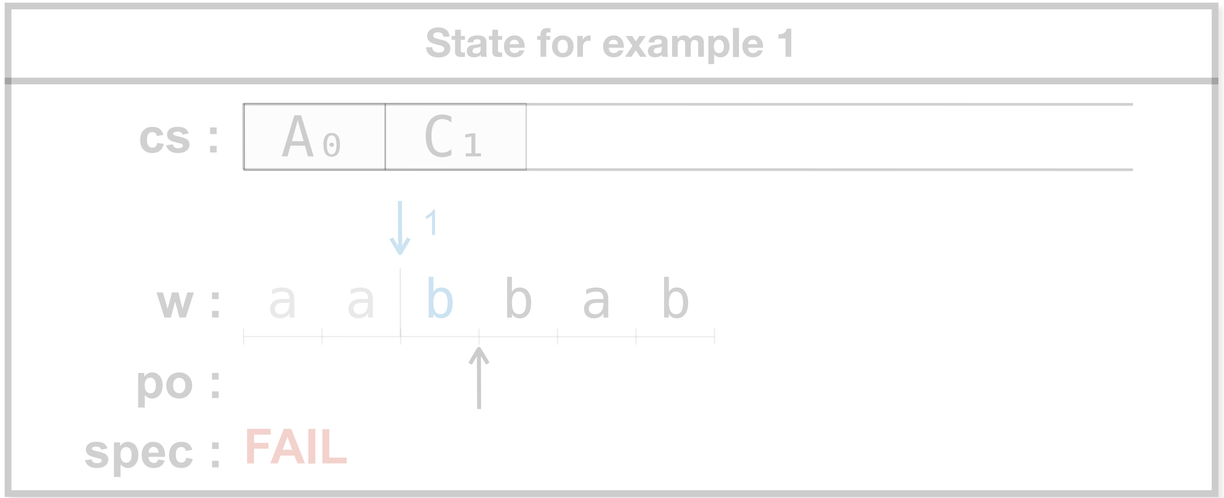
```
A ← BC
    ↑0
B ← (a)*

C ← &(D)

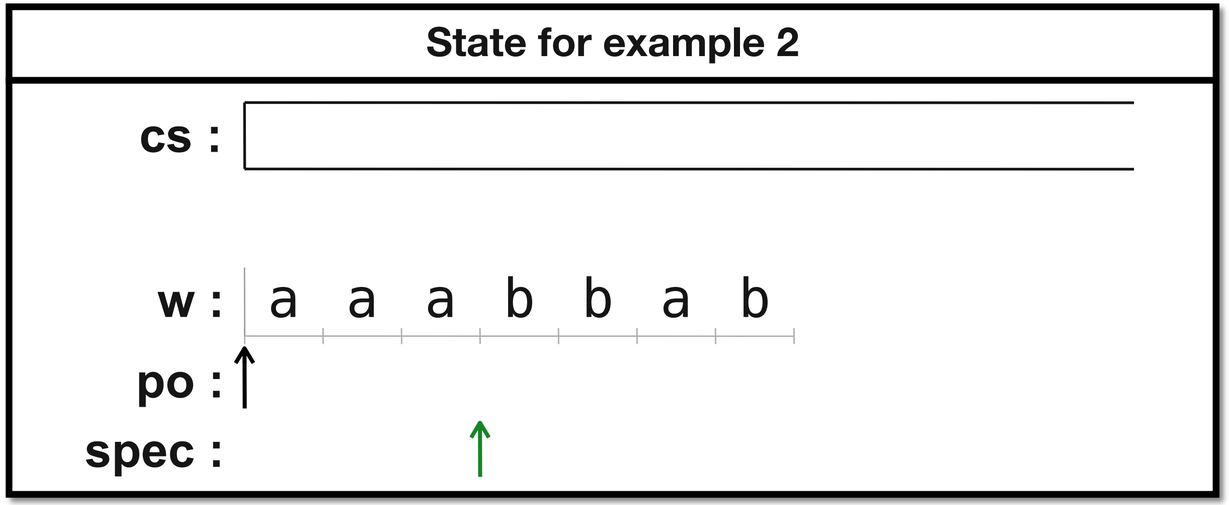
D ← b□
```

Control point ↑ active ↓ suspended

State for example 1



State for example 2

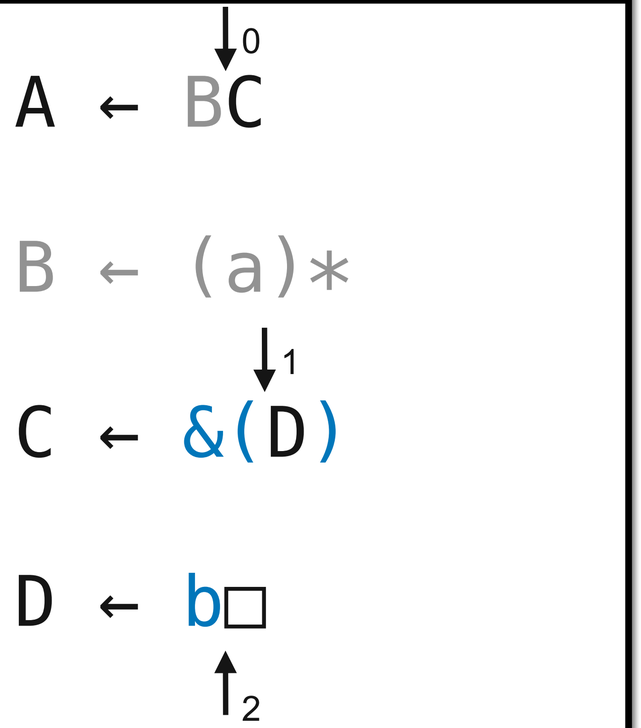


Inconsistent PEG by multiple examples

Given spec

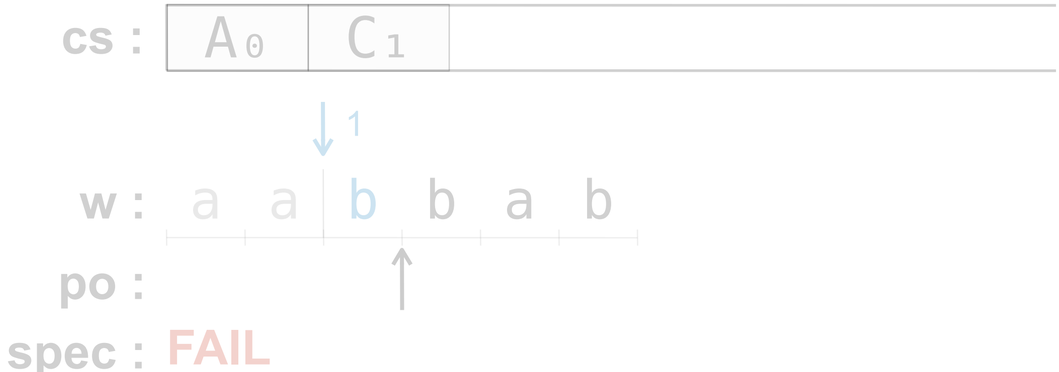
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

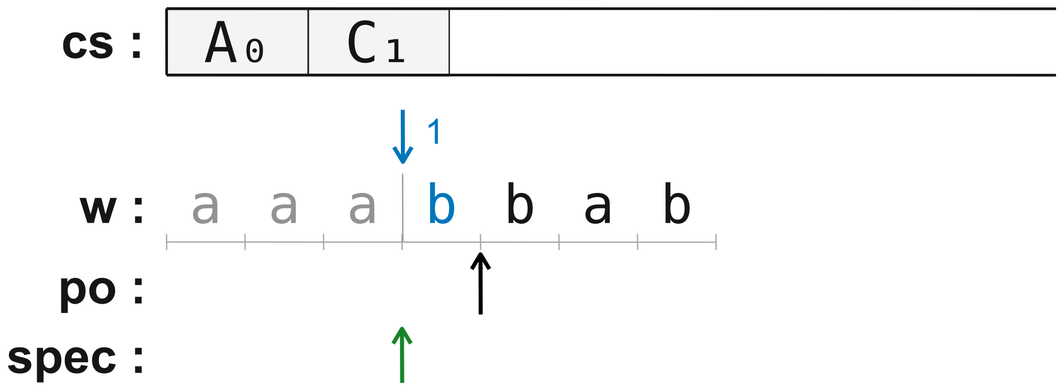


Control point ↑ active ↓ suspended

State for example 1



State for example 2

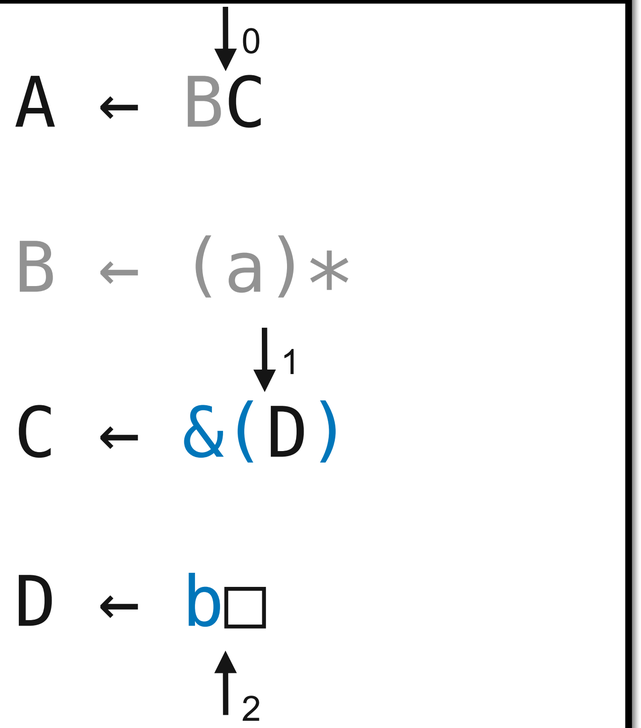


Inconsistent PEG by multiple examples

Given spec

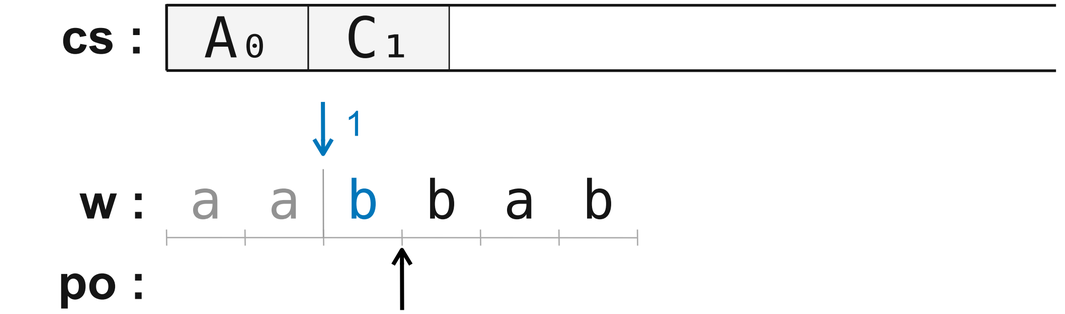
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

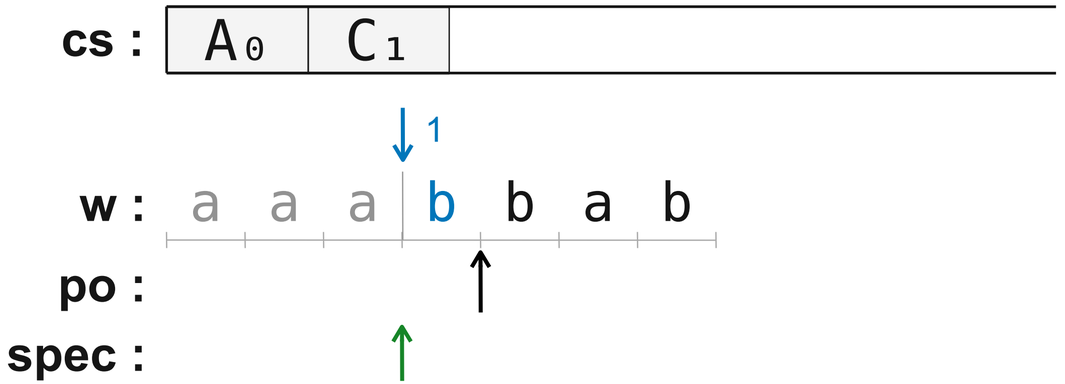


Control point ↑ active ↓ suspended

State for example 1



State for example 2

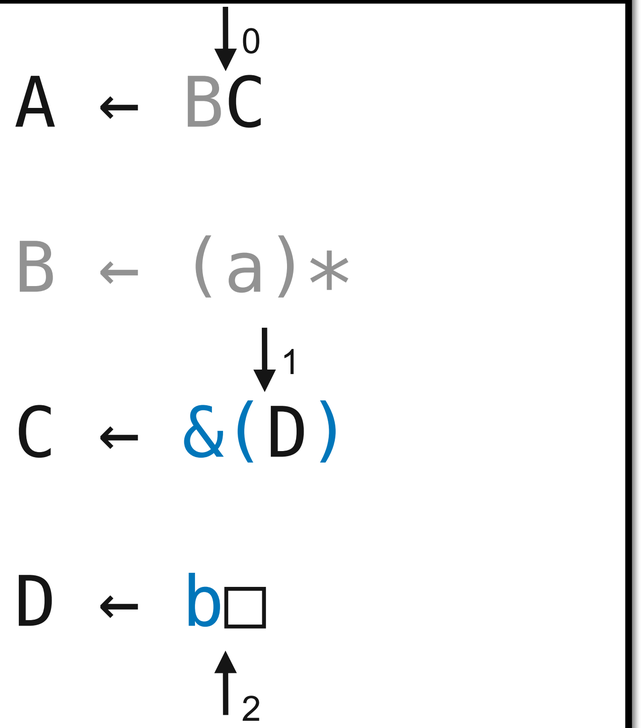


Inconsistent PEG by multiple examples

Given spec

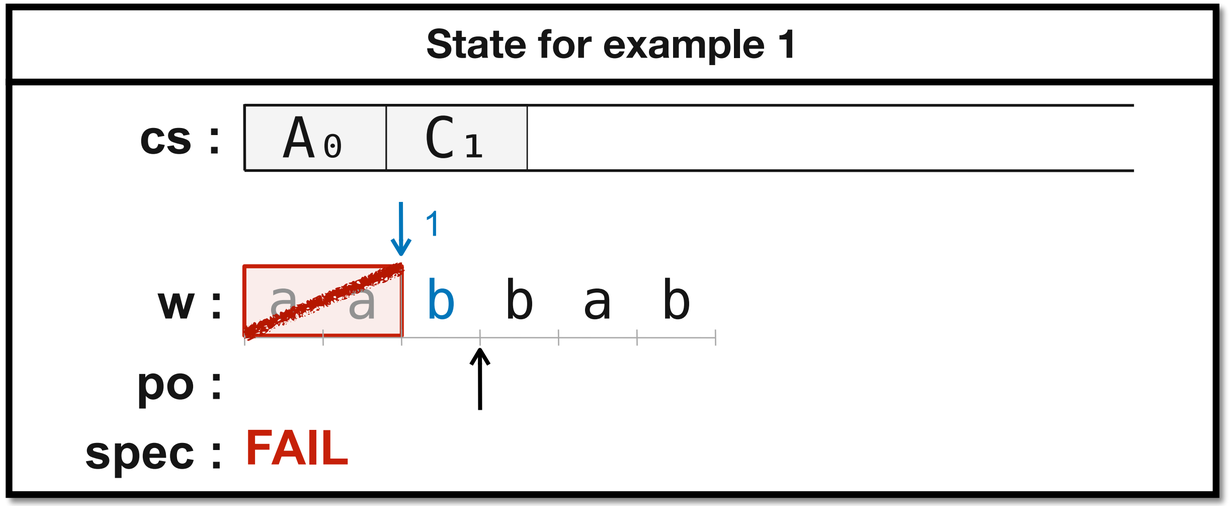
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

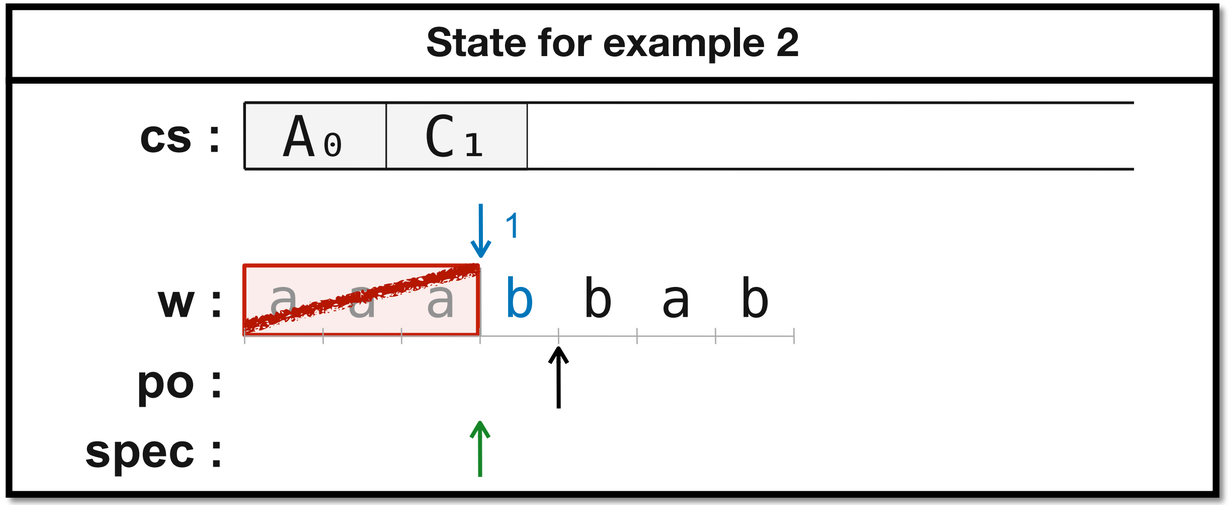


Control point ↑ active ↓ suspended

State for example 1



State for example 2

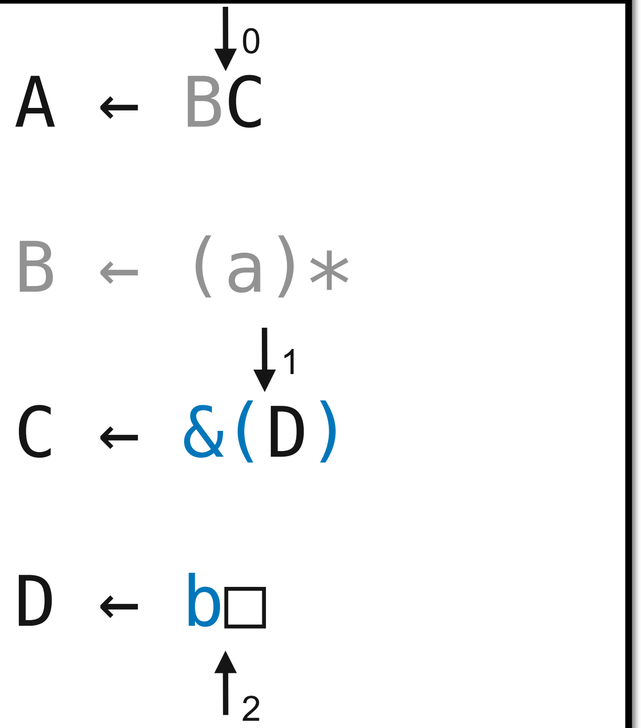


Inconsistent PEG by multiple examples

Given spec

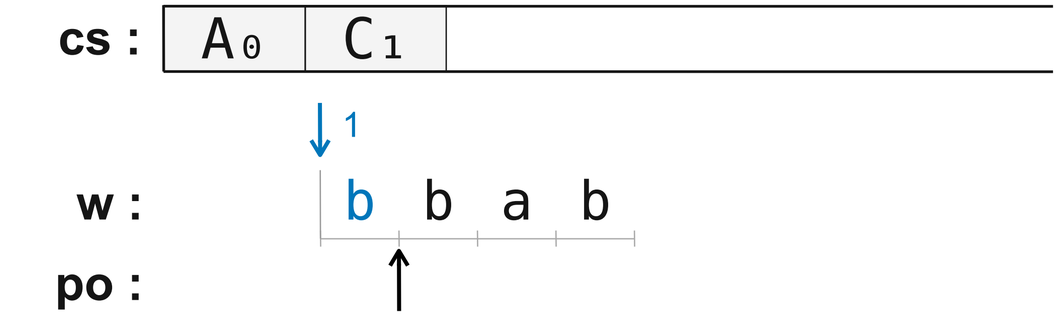
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

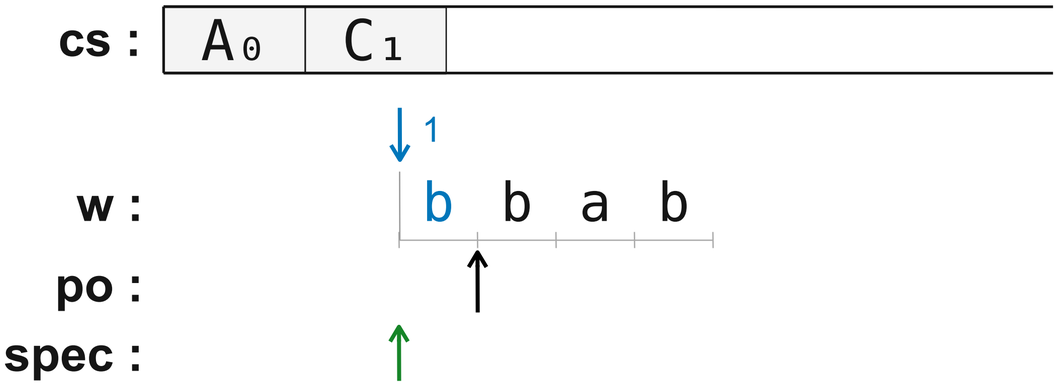


Control point ↑ active ↓ suspended

State for example 1



State for example 2

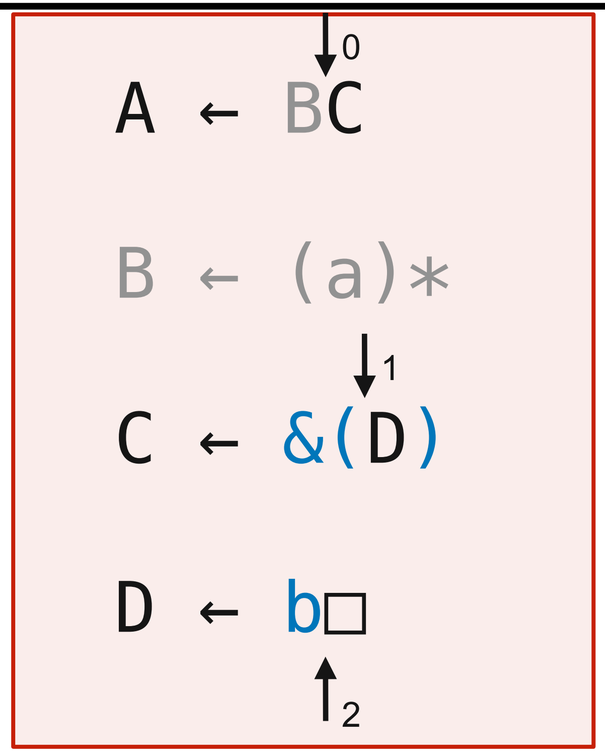


Inconsistent PEG by multiple examples

Given spec

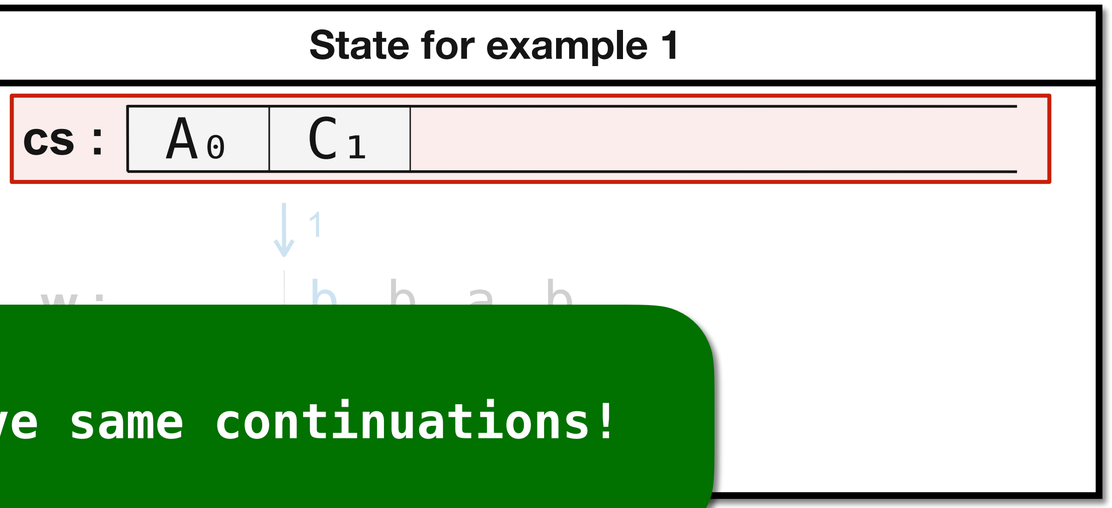
```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4



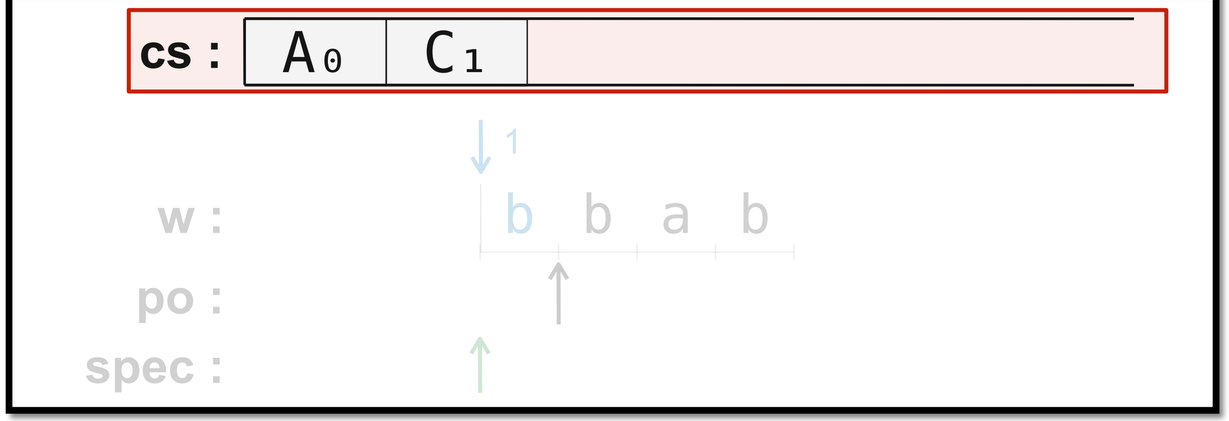
Control point ↑ active ↓ suspended

State for example 1



They have same continuations!

State for example 2



Inconsistent PEG by multiple examples

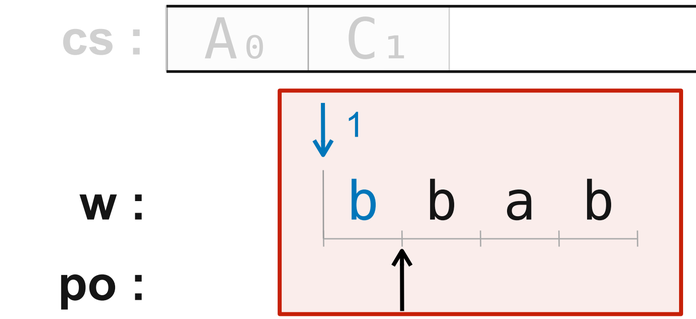
Given spec

```
aabbab → FAIL
aaabbab → bbab
```

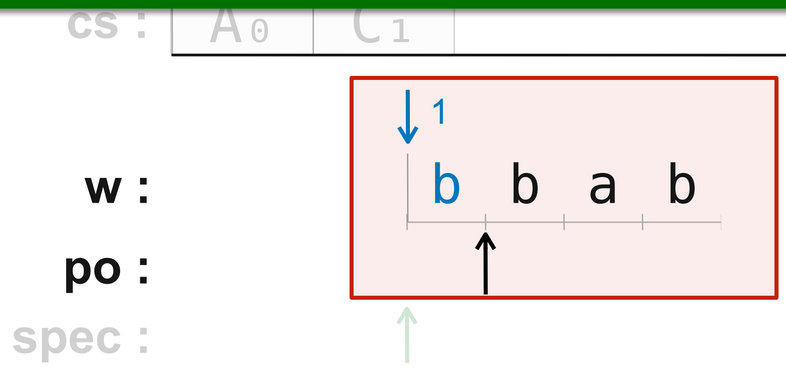
Partial PEG P_4

Control point ↑ active ↓ suspended

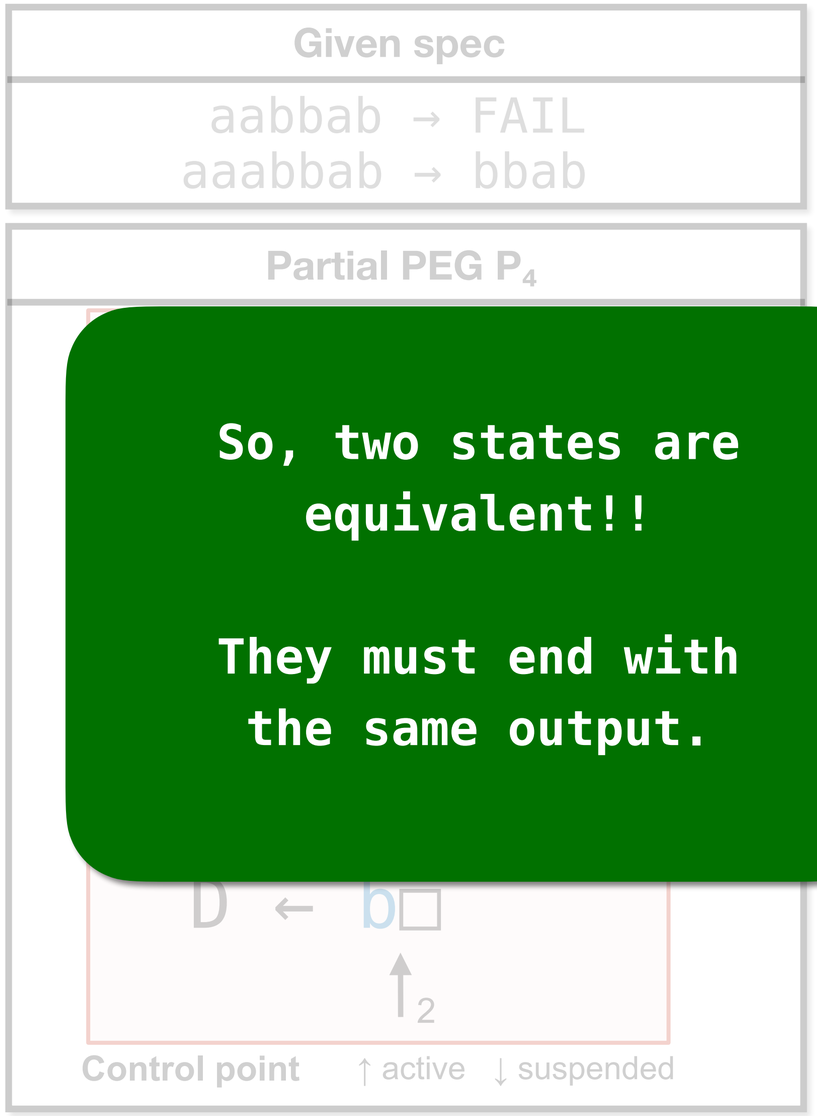
State for example 1



Also have same string and consume states.

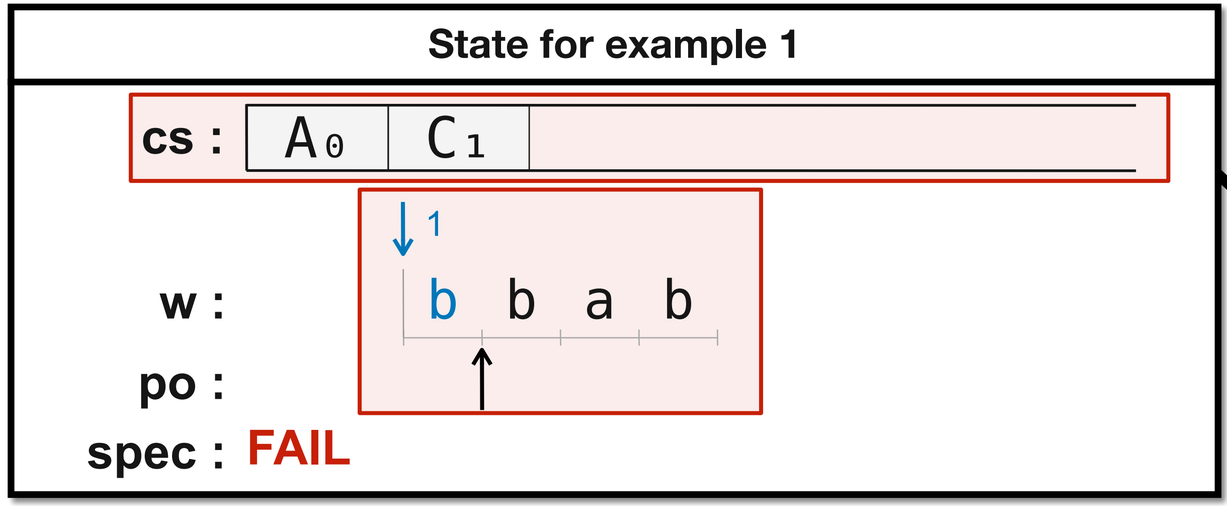


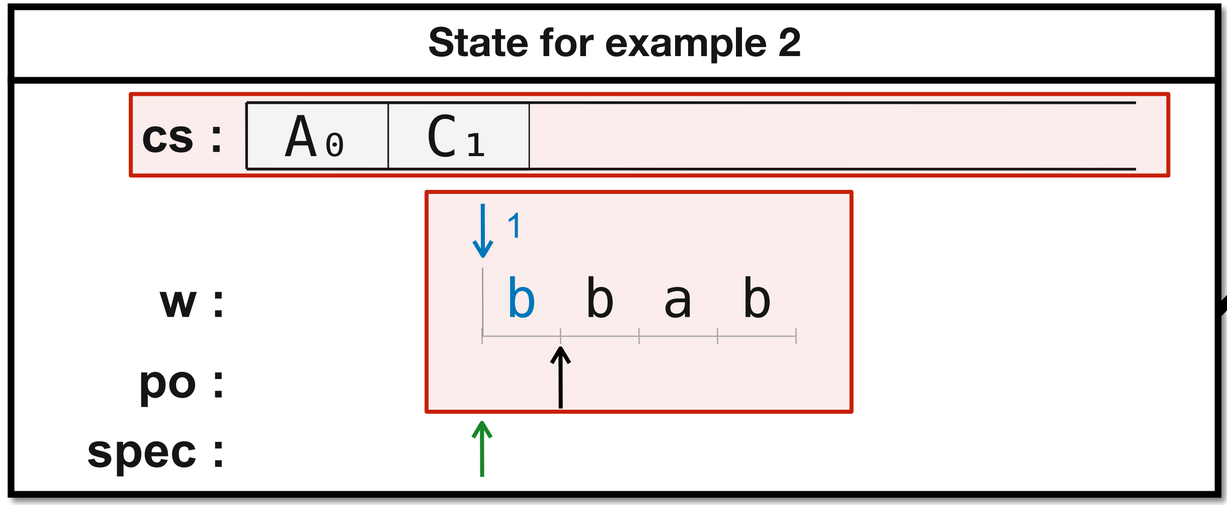
Inconsistent PEG by multiple examples



So, two states are equivalent!!

They must end with the same output.





Inconsistent PEG by multiple examples

Given spec

```
aabbab → FAIL
aaabbab → bbab
```

Partial PEG P_4

Pruning

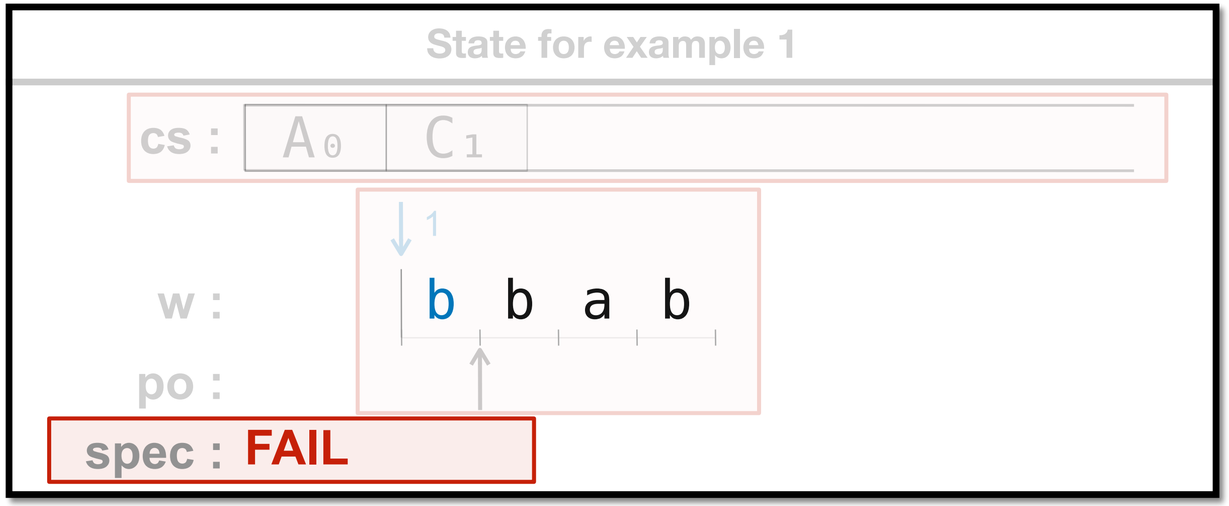
```
A ← BC

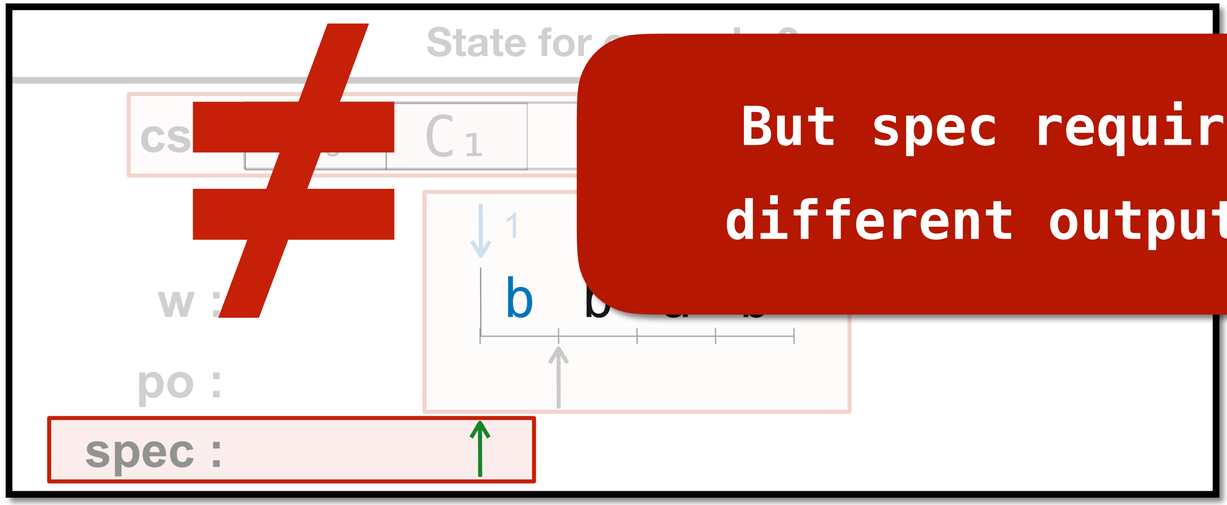
B ← (a)*

C ← &(D)

D ← b□
```

Control point ↑ active ↓ suspended





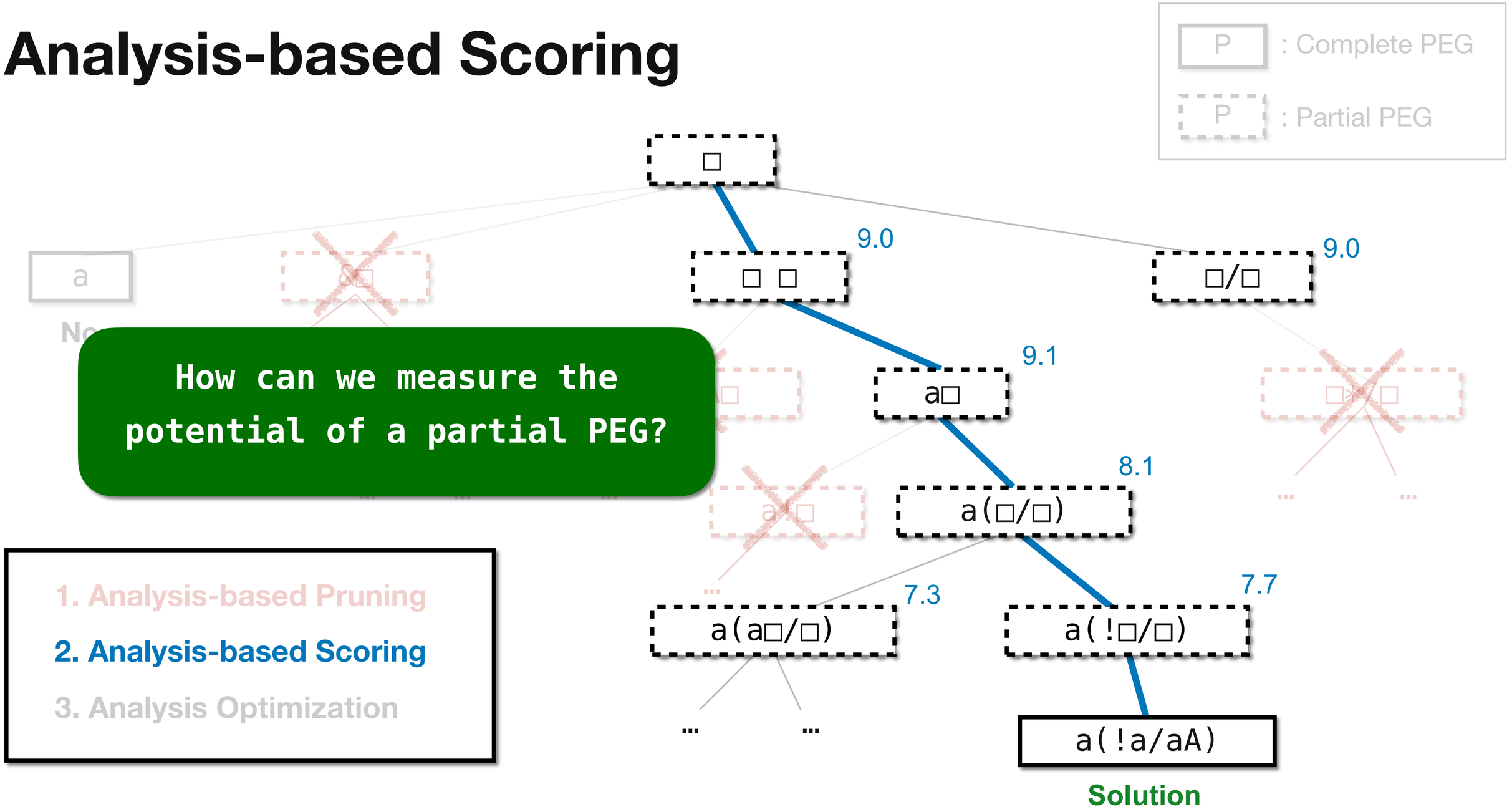
But spec requires different outputs!

Our Approach

- Analysis-based Pruning
  1. Grammar Analysis
     1) Reducible PEG
     2) Left recursive PEG
  2. Example Analysis
     1) Invalid PEG by a single example
     2) Inconsistent PEG by multiple examples
- **Analysis-based Scoring**
- Analysis Optimization

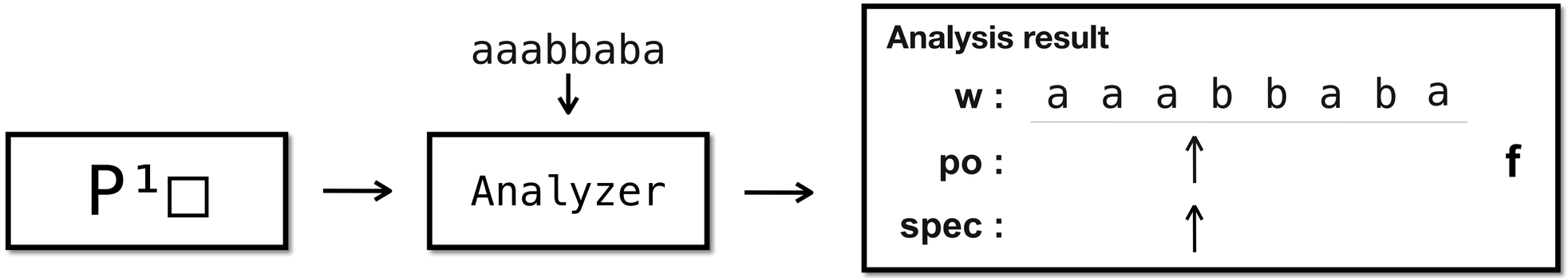
PLRG

Analysis-based Scoring

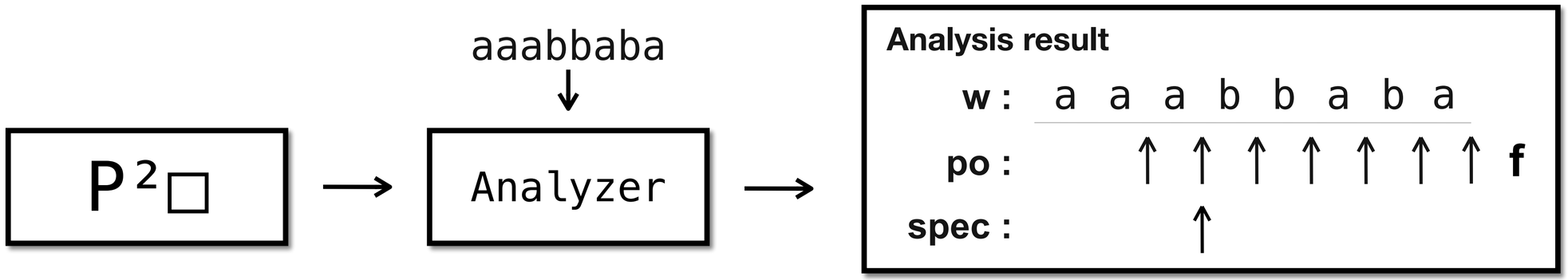


Potential Score

Given spec
aaabbaba → bbaba



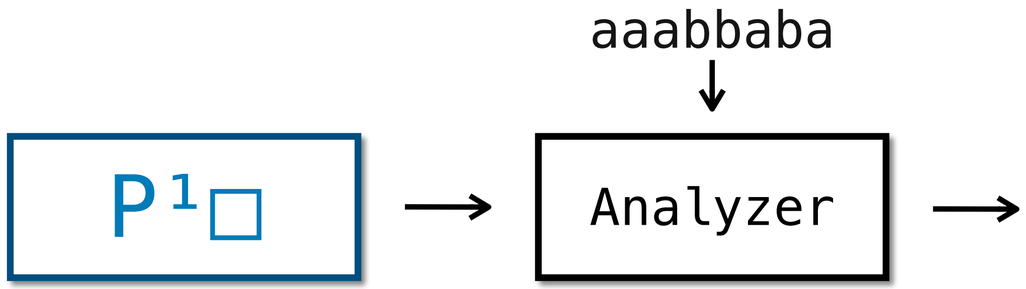
Potential Score: ???



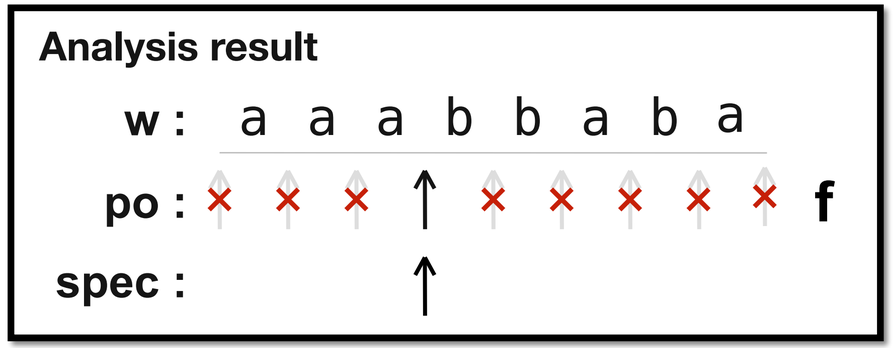
Potential Score: ???

Potential Score

Given spec
aaabbaba → bbaba

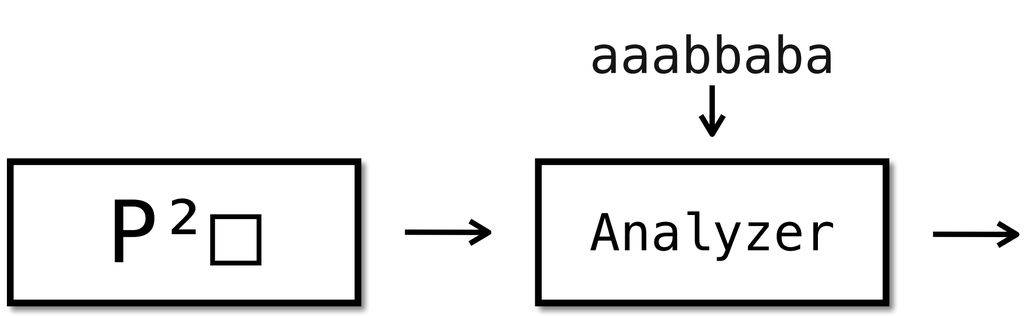


Potential Score: 8

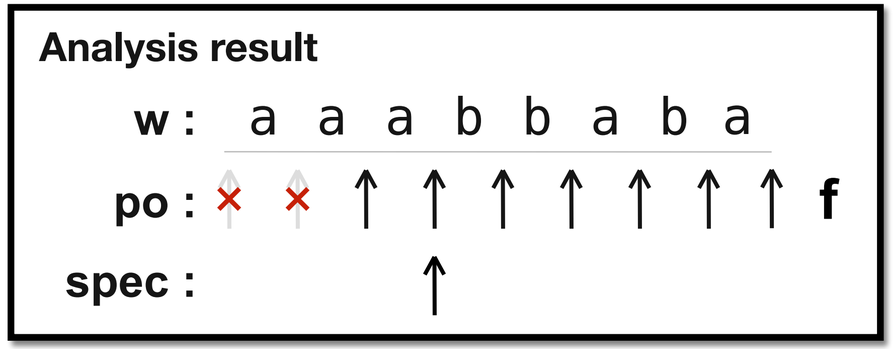


8 wrong outputs filtered

V



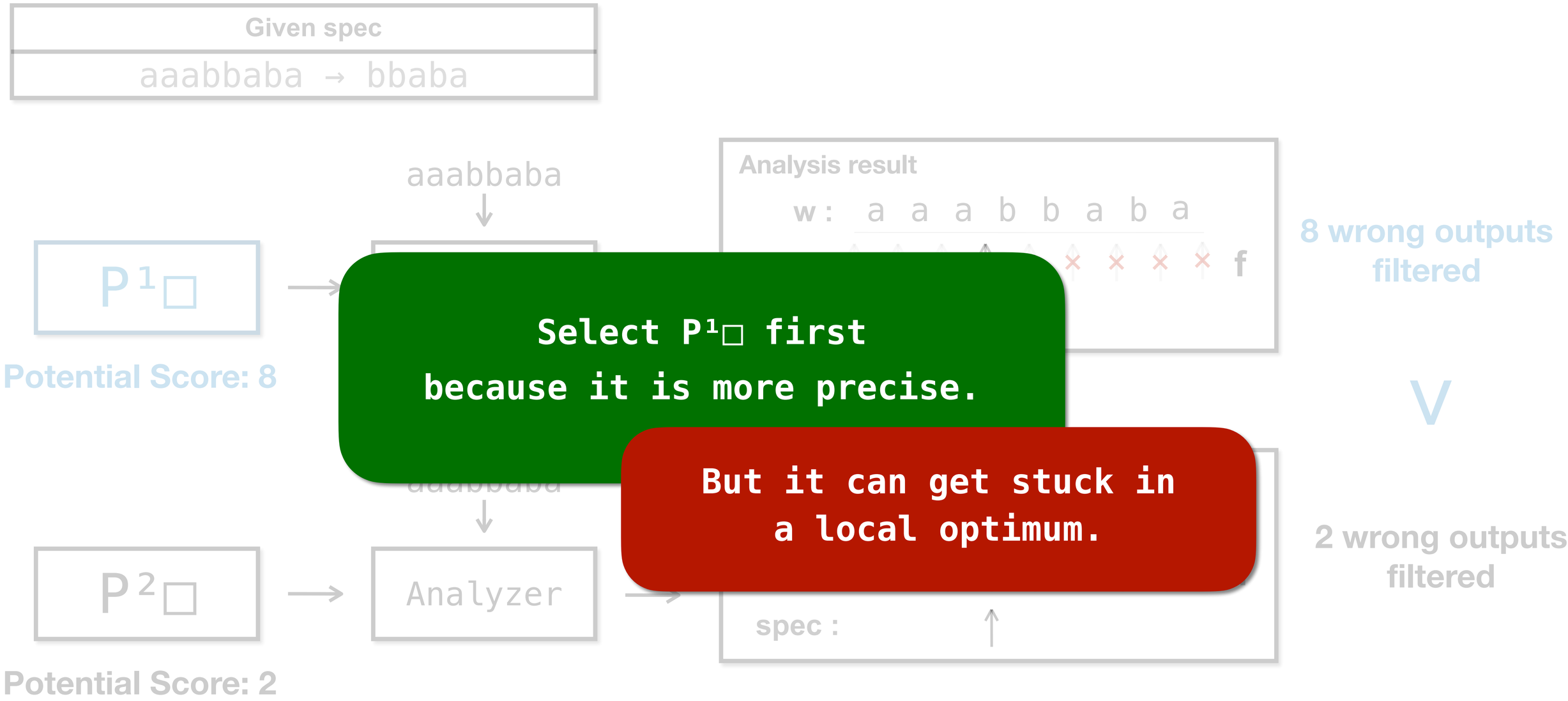
Potential Score: 2



2 wrong outputs filtered

PLRG

Potential Score



Scoring Function

Scoring function for partial PEG P
$f(P) = \alpha \cdot score(P) - size(P)$

Term	Meaning
score(P)	Wrong outputs filtered, normalized over examples
size(P)	Number of operators
α	Weight of the potential score (upper bound of its contribution)

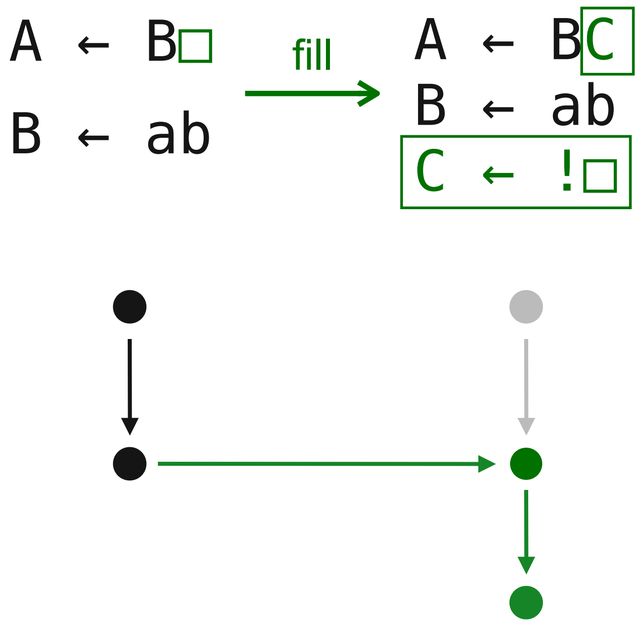
Our Approach

- **Analysis-based Pruning**
  1. **Grammar Analysis**
     1) Reducible PEG
     2) Left recursive PEG
  2. **Example Analysis**
     1) Invalid PEG by a single example
     2) Inconsistent PEG by multiple examples
- **Analysis-based Scoring**
- **Analysis Optimization**

PLRG

Optimization for Analysis

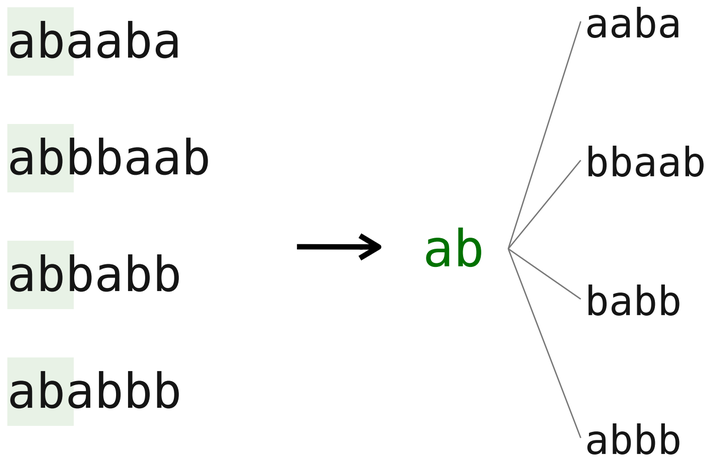
Cross-Grammar



Reuse analysis results

Incremental analysis

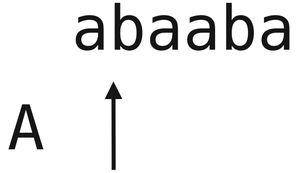
Cross-Example



Example list → example trie

Abstract parsing

Single Example



Memoization table

(A, 1) = o_1

(B, 1) = o_2

…

Abstract packrat parsing

Benchmark

- **15 tasks, from formal specifications to language fragments**

No.	Description	V_T	Predefined Rules	$\#e_{\text{train}}$
1	Parse as many 'a's as possible or 'abb'.	a, b	-	20
2	Parse the leading 'a's only if there is an odd number of 'a's.	a, b	-	20
		...		
15	Parse the keyword 'static' only if it is used for static generators in a JavaScript program.	[a-z], (,), *, {, }, s	WS ← ' '*; NAME ← ALPHA ALPHA*; STATIC ← 'static ' WS; PARAMS ← '()' / '(' NAME (',' NAME)* ')';	30

- **10-30 examples for training, 10,000 examples for test.**

Evaluation: Research Questions

RQ1: How well does SynPEG synthesize PEGs for the benchmarks?

RQ2: How effective are the pruning techniques and the score?

- Search space reduction
- Time speed up

RQ3: How effective are our optimizations?

RQ 1: Synthesis Performance

Method	Size	Time (s)	Accuracy (%)	Solved / 15
LLM	6.3	1.43	75.9	3.4
LLM + desc.	7.5	1.41	74.9	5.4
SynPEG	**4.9**	**0.43**	**100.0**	**15 / 15**

RQ 2: Effectiveness of Techniques

Configuration	Mean candidates	Mean time (s)	Solved / 15
Baseline	>14.91 M	209.37	5
B + Grammar	>9.86 M	166.87	9
G + Example	>1.79 M	97.54	12
E + Score	**17.21 K**	**0.43**	**15**

99.82%

389×

RQ 3: Effectiveness of Optimizations

Analysis	Without Opt. (s)	With Opt. (s)
Grammar	0.76	0.07
Example	0.24	0.06
Total	**1.00**	**0.13**



Future Works

1. **More precise analysis**
   - Prune more partial PEGs

2. **Extend an existing PEG**
   - Using examples for the new behavior

Overall of SynPEG

